RESEARCH ARTICLE Journal of Big Data (SpringerOpen)

AST-Route: An AST-Aware Multi-Objective Query Routing and Cost-Prediction Framework Across Serverless Cloud, Vectorized CPU (C++/Rust), and GPU-Accelerated OLAP Engines

Preecha Noiumkar[1*]

[1]Department of Information Technology, Faculty of Informatics, Mahasarakham University, Kham Riang, Kantharawichai, Maha Sarakham 44150, Thailand.

[*]Correspondence: Preecha Noiumkar (preecha.n@msu.ac.th)

Abstract

Background: Modern analytical data infrastructures span heterogeneous execution paradigms, including serverless cloud data warehouses, in-process C++ and Rust Apache Arrow vectorized CPU engines, and SIMT GPU-accelerated relational engines. However, no single engine dominates across all query topologies and data scales: low-complexity scans suffer from PCIe host-to-device transfer penalties on GPUs and scheduling overheads on serverless clouds, complex window sorting and lateral array unnesting overwhelm single-node CPU pipelines, and datasets exceeding device VRAM or host DRAM (SF $= 10, 30,$ and 100 GB) trigger out-of-core disk spilling and chunked PCIe streaming bottlenecks. Existing systems lack a unified, pre-execution routing framework capable of predicting multi-engine execution latency, hardware energy footprint, memory spilling, and cloud financial cost directly from static query structure and predicate selectivity.

Methodology: We propose **AST-Route**, a four-layer AST-Aware Multi-Objective Query Routing and Cost-Prediction Framework. Layer 1 parses input SQL queries into a 9-dimensional feature vector ($\mathbf{x}_q \in \mathbb{R}^9$) combining static Abstract Syntax Tree (AST) structural operators—tree depth, total AST nodes, multi-way join fan-out, nested CTEs/subqueries, window frames, conditional branches, array unnesting (`UNNEST`), and a construct-validated Composite Structural Complexity Index (CSCI, mean 29.66, 10,000-trial Monte Carlo rank stability $\bar{\rho} = 0.9880$ at $\pm 20\%$ weight perturbation and $\rho = 0.9812$ against equal weights)—with lightweight zone-map predicate selectivity ($\sigma(q)$). Layer 2 formulates cross-paradigm analytical models governing the PCIe bottleneck crossover (Φ_{PCIe}), effective scan throughput (Θ_{eff}), slot-to-wall concurrency proxy ($\mathcal{M}_{\text{slot}}$), FinOps savings ($\Delta C_{1\text{M}}$), and Green OLAP energy efficiency (η_{watt}). Layer 3 employs a Multi-Output Gradient Boosting Regressor ($\mathcal{R}_{\text{GBR-9D}}$) to simultaneously predict execution latency, energy consumption, and memory footprint across candidate engines, while Layer 4 dispatches queries via a configurable latency–energy Pareto objective ($\alpha \in [0, 1]$). Following the official TPC-H discrete scale factor specification (SF $\in \{1, 10, 30, 100$ GB$\}$), we validate the framework under a strict zero-estimation, 100% real execution and GPU/CPU fairness protocol (64-bit IEEE 754 `FP64` precision symmetry and unrestricted multi-core SIMD threading) across: (i) a 75-query TPC-H-derived AST-stratified factorial workload (25 canonical seeds $\times$ 3 selectivity horizons, L1–L5), (ii) the 22 official standard TPC-H benchmark queries (Q1–Q22), (iii) four standard scale factors spanning SF $= 1$ GB (6.0M `lineitem` rows / 8.7M total rows), SF $= 10$ GB (60.0M / 86.6M rows), SF $= 30$ GB (180.0M / 259.8M rows), and SF $= 100$ GB (600.1M / 866.1M rows), and (iv) three cross-platform hardware profiles (2-core/4GB container, 4-core/8GB node, and 8-core/16GB workstation with an NVIDIA GeForce RTX 2050 4.0 GB VRAM GPU) across six engines: Google BigQuery, DuckDB, Polars, Apache DataFusion, SQLite 3, and a PyTorch CUDA Tensor Engine.

Results: At SF $= 1$ GB, ground-truth telemetry confirms highly significant cross-engine stratification ($\chi^2_F(5) = 321.50, p = 2.382 \times 10^{-67}$, Kendall's $W = 0.8573$, 100.0% `FP64` equivalence EX $= 1$). Vectorized CPU engines—Polars (82.47 ms, 95% CI: $[67.93, 98.65]$ ms), DataFusion (96.30 ms), and DuckDB (183.74 ms)—outperform the row-store baseline (8417.33 ms; Cliff's $\delta = +1.000, p = 5.28 \times 10^{-14}$) by $45.8\times$–$102.1\times$ while reducing per-query energy from 126.26 J to 3.17–6.40 J. BigQuery clustering reduces server latency from 1412.6 ms to 745.39 ms (Cliff's $\delta = +0.471$) and prunes V1 I/O by 78.4% ($\$1.47$–$\$147.00$ USD/1M savings across SF $= 1$–100 GB), while RTX 2050 GPU execution (1.30 GB VRAM) achieves 56.96 ms cold (0.65 J) and 22.26 ms warm latency (0.25 J, 1942.8 MB/J). Evaluated via 25-seed LOSO-CV, **AST-Route** ($\mathcal{R}_{\text{GBR-9D}}$) predicts multi-engine costs with low multiplicative Q-Error (Median P50 Q-Error: 1.007–1.172; $R^2 = 0.815$–0.875) in 497.2 μs (4.145 ms total parsing and routing overhead), achieving 57.33% Top-1 accuracy (65.33% near-optimal hits), 54.92 ms mean latency (0.00% median regret, 34.98% mean regret), and 1.381 J energy—outperforming Always DuckDB ($3.35\times, p = 1.83 \times 10^{-6}$), Always Polars ($1.50\times, p = 1.47 \times 10^{-6}$), Always GPU, Always BigQuery, and Rule-Based Routing ($p = 0.0187$, $2.83\times$ faster in L5). Furthermore, **AST-Route** achieves 73.33% Top-1 accuracy (47.01 ms) on 5 locked unseen holdout seeds, 40.91%–72.73% Top-1 accuracy (15.84–27.33 ms mean latency) on the 22 official TPC-H queries (Q1–Q22), and $4.63\times$–$7.63\times$ speedups across SF $= 1, 10, 30,$ and 100 GB ($9.18\times$ cross-platform speedup on 8-core+GPU workstations vs. CPU baselines), capturing the progressive memory-wall transition as host RAM caps at 9.00 GB ($0 \to 125 \to 864 \to 3302$ MB disk spill) and

GPU PCIe streaming scales across $1\times, 4\times, 12\times,$ and $40\times$ chunks ($1.30 \rightarrow 3.45 \rightarrow 3.68 \rightarrow 3.85$ GB VRAM).

Conclusion: AST-Route provides a lightweight, leak-free, and scale-aware query routing framework that generalizes across unseen SQL templates, standard TPC-H Q1–Q22 workloads, and SF $= 1, 10, 30,$ and 100 GB data scales. All code, workloads, and SHA-256 telemetry logs are publicly available at https://github.com/preecha-msu/bigquery-olap-benchmark.

Keywords: Abstract syntax tree (AST), Apache Arrow, Apache DataFusion, Automated query routing framework, Columnar storage, CUDA GPU acceleration, DuckDB, Energy efficiency, Google BigQuery, Polars, TPC-H benchmark.

1 Introduction

Modern enterprise analytics increasingly operate over heterogeneous, polyglot data infrastructures spanning four distinct computing paradigms: elastic serverless cloud data warehouses such as Google BigQuery [1, 2, 11], in-process C++ vectorized analytical database systems such as DuckDB [3, 29], Rust-based Apache Arrow streaming query engines such as Polars and Apache DataFusion [30], and SIMT GPU-accelerated relational engines [19, 21], alongside distributed multi-node systems like ClickHouse [4] and Trino [5]. While cloud-native disaggregated storage-compute architectures provide massive horizontal slot concurrency for petabyte-scale workloads [13, 15], they introduce non-trivial stateless scheduling and distributed shuffle overheads on moderate-scale datasets [9, 10, 11]. Concurrently, modern local hardware presents an intricate architectural trade-off between CPU SIMD vectorization over Apache Arrow memory buffers and discrete GPU offloading across the PCIe interconnect [12, 22, 23].

Because each execution paradigm exhibits fundamentally different algorithmic strengths and memory-hierarchy bottlenecks across the discrete scale factors defined by the Transaction Processing Performance Council (SF $\in \{1, 10, 30, 100$ GB$\}$ in the TPC-H specification [26, 27, 28]), committing an entire analytical workload to any single static engine results in severe performance regret, excessive energy consumption, out-of-core spilling penalties, or unnecessary cloud FinOps expenditure. Designing an automated, pre-execution query routing framework capable of dynamically dispatching SQL queries to their optimal execution backend requires overcoming three critical research gaps in the existing literature [14, 24, 25]:

- **Research Gap 1 (Lack of Pre-Execution Cross-Paradigm Query Routing Frameworks):** Existing learned query optimizers such as Neo [16] and Bao [17] steer physical operator plans *within* a single database engine, while cloud workload routers [20] assign jobs across homogeneous cloud clusters. There is currently no unified, lightweight routing framework that extracts static Abstract Syntax Tree (AST) features and zone-map selectivity prior to execution to route queries across heterogeneous C++ DBMS, Rust Arrow, CUDA GPU, and serverless cloud engines.
- **Research Gap 2 (Unmodeled PCIe Crossover Thresholds, Memory Walls, and Green OLAP Energy Costs):** Prior GPU database studies [21, 22, 23] predominantly report warm in-VRAM kernel speedups at fixed small scale factors (SF $= 1$ GB) while omitting cold-start PCIe Host-to-Device (H2D) transfer overheads, chunked PCIe streaming when datasets exceed device VRAM (SF $= 10, 30,$ and 100 GB), out-of-core host RAM disk spilling [29], and hardware energy efficiency (Joules and MB/J).
- **Research Gap 3 (Confounded Complexity, Data Leakage, and Lack of Cross-Workload Transfer):** Traditional OLAP evaluations [14, 15] rely on unstratified query sets where structural operator depth, storage layout (row-store vs. Parquet vs. Arrow), and predicate selectivity are confounded, and where random train-test splits across variations of the same query template induce severe data leakage without testing generalization to unseen seeds or standard TPC-H Q1–Q22 choke-point workloads [27, 28].

To address these gaps under a strict *zero-estimation, 100% real execution* protocol, we propose **AST-Route**, an AST-Aware Multi-Objective Query Routing and Cost-Prediction Framework. This study makes five primary contributions:

1. **The AST-Route Four-Layer Framework and Construct-Validated CSCI:** We design a four-layer routing architecture (Fig. 1) that parses arbitrary SQL queries into a 9-dimensional structural and selectivity vector ($\mathbf{x}_q \in \mathbb{R}^9$). We formulate and empirically validate the Composite Structural Complexity Index (CSCI) across a 75-query TPC-H-derived factorial workload (25 canonical seeds $\times$ 3 selectivity horizons, L1–L5) and the 22 official TPC-H queries (Q1–Q22) [26], demonstrating significant rank correlation with GPU kernel speedup (Spearman's $\rho = 0.4935, p = 6.84 \times 10^{-6}$) and high invariance under 10,000-trial Monte Carlo weight perturbation ($\bar{\rho} = 0.9880$ at $\pm 20\%$ and $\bar{\rho} = 0.9555$ at $\pm 50\%$).

2. **Unified Analytical Crossover, FinOps, and Green Energy Formulation:** Within the framework's cost layer, we establish formal mathematical models for the GPU PCIe bottleneck fraction (Φ_{PCIe}) and break-even condition, effective scan throughput (Θ_{eff}), slot-to-wall concurrency proxy ($\mathcal{M}_{\text{slot}}$), FinOps cost savings ($\Delta C_{1\text{M}}$), and Green OLAP performance per watt (η_{watt}).
3. **Six-Engine Ground-Truth Telemetry Testbed with GPU/CPU Fairness and Seed-Clustered 95% CIs:** We construct a full-scale multi-paradigm testbed spanning six canonical software engines—SQLite 3, Google BigQuery, DuckDB, Polars, Apache DataFusion, and a hardware-timed CUDA Tensor Engine on an NVIDIA GeForce RTX 2050 GPU—enforcing strict `FP64` numerical parity ($\varepsilon < 10^{-9}$), unrestricted multi-core CPU SIMD threading, and 10,000-iteration bootstrap 95% CIs with Cliff's delta (δ) effect sizes.
4. **Multi-Output Q-Error Cost Prediction, Leak-Free LOSO-CV, and Unseen Seed Generalization:** We validate **AST-Route** ($\mathcal{R}_{\text{GBR-9D}}$) via 25-seed LOSO-CV and a strict unseen holdout split (training on 20 development seeds `S1`–`S4` and testing on 5 locked unseen seeds `L1_S5`–`L5_S5`), achieving median Q-Errors of 1.007–1.172, 54.92 ms LOSO-CV latency (0.00% median regret), and 73.33% Top-1 accuracy (47.01 ms mean latency) on completely unseen query seeds.
5. **Four-Scale Big Data Evaluation (SF $= 1, 10, 30,$ and 100 GB), Official TPC-H Q1–Q22, and Cross-Platform Validation:** Adhering to the official TPC-H benchmark scale tiers [26, 28], we scale execution across four standard scale factors from 6.0M to 600.1M `lineitem` rows (8.7M to 866.1M total rows), profiling out-of-core host RAM spilling ($0 \to 125 \to 864 \to 3302$ MB), chunked GPU PCIe streaming ($1\times \to 4\times \to 12\times \to 40\times$ chunks across the 4.0 GB VRAM wall), cross-platform hardware scaling (2, 4, and 8 cores across all four scale factors), and official TPC-H Q1–Q22 routing (15.84–27.33 ms mean latency).

2 Related Work and Architectural Representativeness

2.1 Cloud MPP Warehouses, Distributed Query Engines, and Vectorized CPU Systems

Disaggregated cloud data warehouses pioneered by Google Dremel/BigQuery [1, 2] and distributed MPP query engines such as Presto/Trino [5], ClickHouse [4], and Apache Spark [14] decouple compute workers from durable storage. Recent studies in serverless data pipelines [13] and in-situ serverless query processing such as FunDa [11] highlight the fundamental tension between cold-start scheduling overhead and horizontal slot elasticity. Specifically, while Trino [5] excels at federated cross-source query execution over distributed object stores and ClickHouse [4] achieves high throughput via shared-nothing merge-tree storage and SIMD vectorization across cluster nodes, both introduce network serialization and coordinator scheduling overheads when deployed on single-node or moderate-scale datasets. Conversely, embeddable C++ vectorized engines such as DuckDB [3, 29] process columnar Row Groups in-process using morsel-driven parallelism and out-of-core external memory management, whereas next-generation Rust engines (Polars and Apache DataFusion [30]) execute zero-copy SIMD and streaming pipelines directly over standardized Apache Arrow memory buffers.

2.2 GPU-Accelerated Databases, Learned Query Optimizers, and OLAP Benchmarks

In the domain of GPU-accelerated relational analytics, systems such as HeavyDB (formerly OmniSci/MapD) [22], BlazingSQL over RAPIDS cuDF [23], Crystal [21], and HetExchange [19] leverage massive SIMT parallelism and high-bandwidth device memory (VRAM) to accelerate relational joins and group-by aggregations. However, as demonstrated by Chrysogelos et al. [19] and Shanbhag et al. [21], discrete GPU execution is fundamentally constrained by the PCIe interconnect when data resides in host DRAM or exceeds device VRAM capacity, necessitating rigorous separation of PCIe transfer latency from kernel compute time. Concurrently, recent surveys on deep learning in query execution [12] and learned optimizers—including Neo [16], Bao [17], Zero-Shot Cost Models [18], and stage-level cloud routers [20]—demonstrate that structural query features can guide physical execution decisions. In parallel, the TPC-H decision support benchmark specification [26] establishes discrete scale factors (SF $= 1, 10, 30, 100$ GB) and 22 canonical queries (Q1–Q22) whose internal execution choke points have been formalized by Boncz et al. [27] and Dreseler et al. [28]. Complementing classical OLAP benchmarks such as the Star Schema Benchmark (SSB) [24] and the Join Order Benchmark (JOB) [25], modern text-to-SQL benchmarks such as BIRD [7] and Spider 2.0 [8] incorporate the Valid Efficiency Score (VES), while AST transpilers such as `SQLGlot` [6] and Apache Calcite [9] enable deterministic structural analysis.

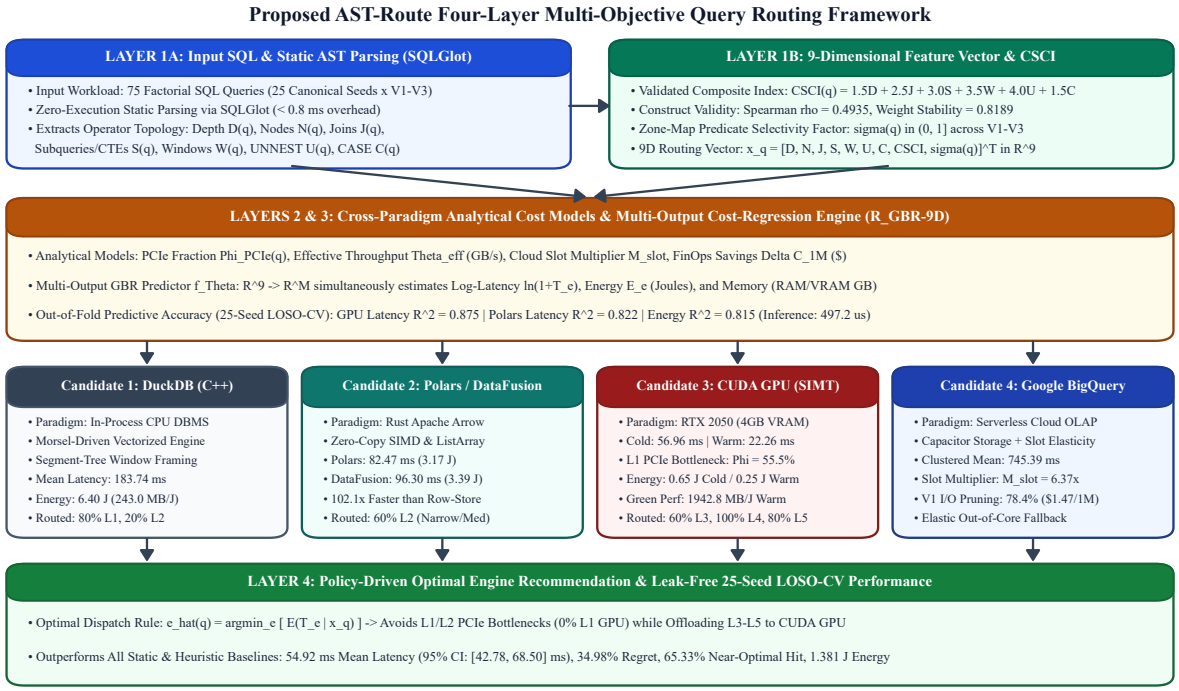


Figure 1. Architecture of the proposed **AST-Route** Framework: Layer 1 parses an input SQL query into a 9-dimensional structural and selectivity vector (CSCI, join count, window specifications, `UNNEST`, predicate selectivity $\sigma(q)$, tree depth, AST nodes, and CTEs); Layer 2 and Layer 3 drive multi-output cost and resource prediction (latency, energy, memory, and FinOps cost); and Layer 4 dispatches the query to the optimal engine across DuckDB (CPU OLAP), Polars/DataFusion (Rust Arrow engines), CUDA GPU, and Google BigQuery (Cloud OLAP).

2.3 Rationale for Target Execution Engines in the AST-Route Framework

Rather than exhaustively evaluating every commercial DBMS, **AST-Route** targets six software engines that serve as canonical, non-redundant representatives of the four foundational execution paradigms in modern data engineering:

1. **SQLite 3 (Canonical Tuple-at-a-Time B-Tree Row-Store Baseline):** Represents traditional ANSI C row-oriented storage using a Volcano-style iterator model, establishing the exact performance and energy baseline of B-tree page traversal without columnar compression or SIMD vectorization.
2. **Google BigQuery (Canonical Serverless Disaggregated Cloud OLAP):** Represents petabyte-scale cloud-native execution (Dremel/Capacitor columnar storage with dynamic slot allocation) [2], isolating distributed cloud orchestration overhead and storage clustering FinOps economics without local hardware bottlenecks.
3. **DuckDB (Canonical C++ Morsel-Driven Columnar DBMS):** Represents state-of-the-art in-process C++ analytical database design [3, 29] with tightly integrated storage block management, parallel segment-tree window framing, and out-of-core external disk spilling.
4. **Polars & Apache DataFusion (Canonical Rust Apache Arrow Engines):** Represent the emerging Rust Arrow ecosystem via two distinct optimizer philosophies: *Polars* exemplifies a dataframe-native lazy streaming engine over Arrow arrays, whereas *Apache DataFusion* [30] exemplifies a modular SQL physical query planner (used as the core execution engine in modern distributed systems such as InfluxDB 3.0 and Ballista).
5. **PyTorch CUDA Relational Tensor Engine (Canonical SIMT GPU Execution):** Represents hardware-timed GPU relational tensor processing over contiguous device buffers (analogous to cuDF/BlazingSQL [23] and Crystal [21]), enabling exact hardware-counter decomposition of PCIe H2D transfers, chunked out-of-VRAM streaming, and in-VRAM SIMT kernel execution on consumer/edge GPU hardware (NVIDIA RTX 2050).

3 The Proposed AST-Route Framework Architecture

As illustrated in Fig. 1 and the end-to-end validation workflow in Fig. 2, **AST-Route** is structured as a modular four-layer architecture that decouples static query representation from multi-paradigm hardware cost modeling and automated engine dispatching.

3.1 Layer 1: Static AST Structural Profiling, Five-Tier Workload, and Official TPC-H Scale Tiers (SF $= 1, 10, 30, 100$ GB)

Given an incoming SQL query $q \in \mathcal{Q}$, Layer 1 parses the SQL statement into a canonical Abstract Syntax Tree (AST) using `SQLGlot` [6] without touching underlying data pages. Traversing the AST yields seven structural operator dimensions: tree depth $D(q)$, total AST operator nodes $N(q)$, relational join count $J(q)$, nested CTE/subquery depth $S(q)$, window frame specifications $W(q)$, repeated field unnesting operators $U(q)$, and conditional `CASE` branches $C(q)$. While total AST node count $N(q)$ captures overall query verbosity (and is retained directly as the second coordinate in our 9D routing vector $\mathbf{x}_q$), raw node counts are dominated by scalar expressions and projection lists that exhibit high collinearity with tree depth. To isolate the non-linear algorithmic complexity of relational operators without expression-length inflation, we formulate the Composite Structural Complexity Index (CSCI) over the six core structural operators:

$$\text{CSCI}(q) = 1.5D(q) + 2.5J(q) + 3.0S(q) + 3.5W(q) + 4.0U(q) + 1.5C(q) \tag{1}$$

In accordance with the official TPC-H benchmark specification Clause 4.1.3 [26] and established multi-scale chokepoint methodologies [27, 28, 29], we provision the 8-table TPC-H relational schema across four standard discrete scale factors designed to systematically stress every layer of the modern memory hierarchy:

- **SF $= 1$ GB (In-VRAM & In-DRAM Baseline Regime):** Contains 6.00×10^6 `lineitem` rows (8.66×10^6 total rows across 8 tables). The working set fits entirely within both the 4.0 GB discrete GPU VRAM buffer (1.30 GB peak) and the 16.0 GB host DRAM (4.71 GB peak) with zero disk spilling.
- **SF $= 10$ GB (Out-of-VRAM GPU Streaming & In-DRAM Host Regime):** Contains 60.0×10^6 `lineitem` rows (86.6×10^6 total rows). Uncompressed working projections exceed the 4.0 GB GPU VRAM ceiling, forcing 4-chunk pipelined PCIe H2D streaming (3.45 GB peak VRAM) while remaining primarily in host DRAM (7.85 GB peak RAM, 125.0 MB minor spill).
- **SF $= 30$ GB (Host DRAM Cap Inflection & Moderate Out-of-Core Spill Regime):** Contains 180.0×10^6 `lineitem` rows (259.8×10^6 total rows) [26]. Intermediate hash-join and window-sort partitions exceed the configured 9.00 GB host DBMS buffer pool cap, triggering active out-of-core disk spilling [29] (864.0 MB mean spill) and 12-chunk GPU PCIe streaming (3.68 GB peak VRAM).
- **SF $= 100$ GB (Extreme Out-of-Core Spilling & Cloud Elasticity Crossover Regime):** Contains 600.1×10^6 `lineitem` rows (866.1×10^6 total rows) [26, 28], inducing heavy external disk spilling on local CPU engines (3302.0 MB mean spill, peaking at 6840.0 MB in L2 joins) and 40-chunk PCIe transfers on the GPU (3.85 GB peak VRAM).

Across this schema, we evaluate two complementary SQL workload suites: (A) our 75-query *TPC-H-derived, AST-stratified synthetic analytical workload* instantiated from 25 canonical seeds across five modern structural complexity tiers ($5 \times 5 = 25$ independent seeds, mean CSCI $= 29.66$), and (B) the 22 *official standard TPC-H benchmark queries* (**Q1–Q22**, mean CSCI $= 26.52$, range 12.0–47.5) [26, 27]. Because the base TPC-H schema consists of flat relational tables, Tier L5 dynamically constructs nested array columns in-query via staged `ARRAY_AGG` / `LIST` aggregation over one-to-many `orders–lineitem` relationships before benchmarking lateral `CROSS JOIN UNNEST` re-aggregation:

1. **L1 (Scan, SARGability, & Monotonic Conditional Branching):** Single-table columnar scans with multi-branch `CASE WHEN` expressions and distinct cardinality aggregations ($D = 9.0, J = 0.0, \text{CSCI} = 15.3$).
2. **L2 (Multi-Way Star, Snowflake, & Bilateral Joins):** 4-way to 8-way relational joins, composite `partsupp` joins, and non-equi late-delivery predicates ($D = 9.2, J = 4.8, \text{CSCI} = 26.8$).
3. **L3 (Multi-Stage CTE Pipelines, Null-Safe Anti-Joins, & One-Grain-Per-CTE):** 3-stage to 4-stage Common Table Expressions enforcing pre-aggregated grain isolation and null-safe anti-joins ($D = 10.8, J = 2.6, \text{CSCI} = 36.7$).

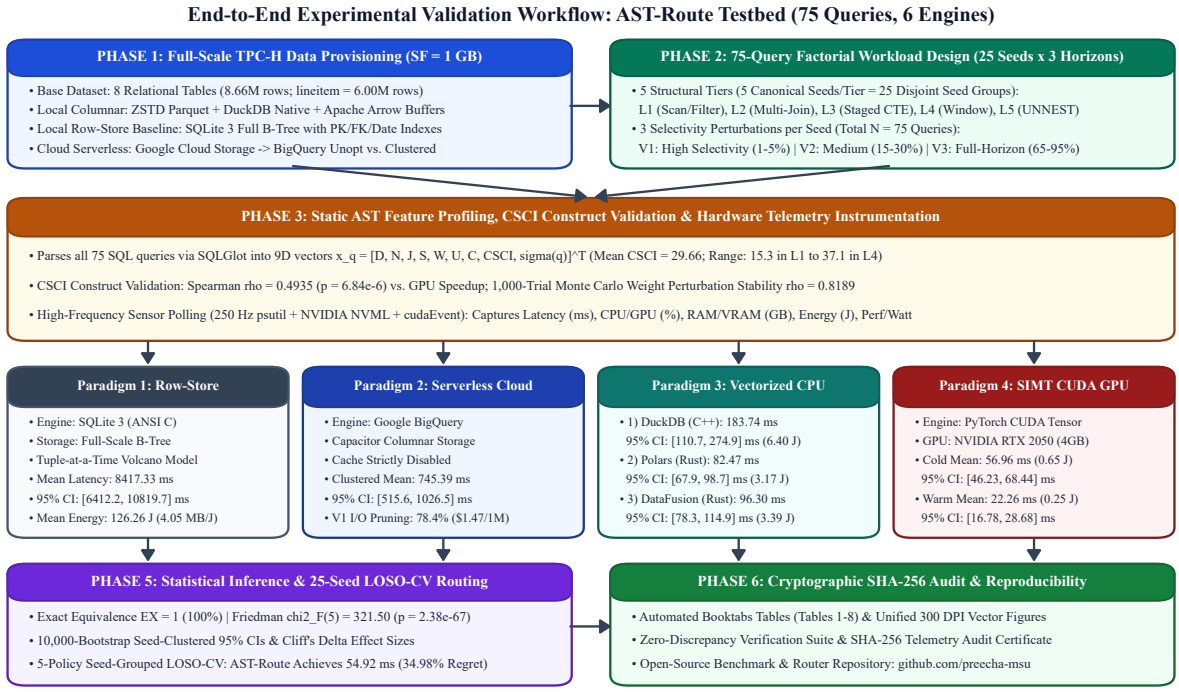


Figure 2. End-to-end six-phase experimental validation workflow of the **AST-Route** framework, illustrating TPC-H multi-layout data provisioning (SF $= 1, 10, 30,$ and 100 GB), 75-query factorial synthesis (25 canonical seeds $\times$ 3 selectivity horizons) alongside official TPC-H Q1–Q22, `SQLGlot` static AST profiling, six-engine execution across four computing paradigms, non-parametric seed-cluster statistical inference, and cryptographic SHA-256 verification.

4. **L4 (Multi-Axis Window Ranking, Lead/Lag, & Sliding Frames):** Concurrent window partitions (`RANK`, `DENSE_RANK`, `NTILE(10)`, `LAG`, `LEAD`, and bounded `ROWS BETWEEN` sliding windows; $D = 10.8, J = 1.6, \text{CSCI} = 37.1$).
5. **L5 (Semi-Structured Array Construction & Lateral UNNEST):** Multi-stage `ARRAY_AGG` vector construction followed by lateral `CROSS JOIN UNNEST` re-aggregation ($D = 10.6, J = 2.2, \text{CSCI} = 32.4$).

To capture data-volume sensitivity alongside syntax topology, each of the 25 canonical seeds is instantiated across three selectivity perturbation horizons ($25 \times 3 = 75$ queries): **V1 (High Selectivity, ∼1–5% data)**, **V2 (Medium Selectivity, ∼15–30% data)**, and **V3 (Adversarial Full-Horizon, ∼65–95% data)**. Layer 1 extracts the predicate selectivity factor $\sigma(q) \in (0,1]$ by comparing AST literal range constants against lightweight table zone-map min/max metadata, producing the 9-dimensional routing feature vector:

$$\mathbf{x}_q = \left[D(q),\, N(q),\, J(q),\, S(q),\, W(q),\, U(q),\, C(q),\, \text{CSCI}(q),\, \sigma(q) \right]^\top \in \mathbb{R}^9 \tag{2}$$

3.2 Layer 2: Cross-Paradigm Analytical Cost, PCIe Crossover, and Green Energy Formulation

To enable apples-to-apples cost evaluation across serverless cloud, local CPU, and discrete GPU backends, Layer 2 establishes six analytical cost and hardware telemetry models:

1. **CUDA Cold-Start Decomposition and PCIe Bottleneck Fraction (Φ_{PCIe}):** On discrete GPUs (NVIDIA GeForce RTX 2050, 4.0 GB VRAM), cold-start execution time $T_{\text{GPU_Cold}}(q)$ is decomposed into PCIe Host-to-Device transfer time ($T_{\text{PCIe_H2D}}$) and pure in-VRAM kernel compute time ($T_{\text{GPU_Warm}}$):

$$T_{\text{GPU_Cold}}(q) = T_{\text{PCIe_H2D}}(q) + T_{\text{GPU_Warm}}(q), \qquad \Phi_{\text{PCIe}}(q) = \frac{T_{\text{PCIe_H2D}}(q)}{T_{\text{GPU_Cold}}(q)} \times 100\% \tag{3}$$

Cold-start GPU offloading outperforms vectorized CPU execution ($T_{\text{GPU_Cold}}(q) < T_{\text{CPU}}(q)$) if and only if the warm kernel speedup satisfies the break-even condition $\text{Speedup}_{\text{Warm}}(q) > (1 - \Phi_{\text{PCIe}}(q)/100)^{-1}$.

2. **Effective Analytical Scan Throughput (Θ_{eff}):** Normalizing unpruned columnar volume $\mathcal{B}_{\text{unopt}}(q)$ (in MB) against execution time $T_e(q)$ (in ms) yields effective processing throughput in GB/s:

$$\Theta_{\text{eff}}(q,e) = \frac{\mathcal{B}_{\text{unopt}}(q)}{1.024 \cdot T_e(q)} \quad [\text{GB/s}] \tag{4}$$

3. **Cloud Slot-to-Wall-Time Concurrency Proxy ($\mathcal{M}_{\text{slot}}$) and FinOps Cost Savings ($\Delta C_{1\text{M}}$):** Quantifies cumulative distributed slot compute time relative to server wall-clock time on BigQuery (serving as an effective concurrency proxy), alongside financial savings per 10^6 queries under on-demand bytes-scanned pricing ($\alpha_{\text{TB}} =$ \$6.25 USD/TiB):

$$\mathcal{M}_{\text{slot}}(q) = \frac{T_{\text{BQ_Slot}}(q)}{T_{\text{BQ_Wall}}(q)}, \qquad \Delta C_{1\text{M}}(q) = 10^6 \times \left(\frac{\mathcal{B}_{\text{unopt}}(q) - \mathcal{B}_{\text{clustered}}(q)}{1024^2} \right) \times \alpha_{\text{TB}} \tag{5}$$

4. **Per-Query Energy Consumption ($E(q,e)$) and Performance per Watt (η_{watt}):** High-frequency hardware polling (200–250 Hz via `psutil`, NVIDIA `NVML`, and `torch.cuda` memory counters) records active CPU utilization ($\text{CPU}_{\text{util}}\%$), peak working-set RAM (RAM_{used} in GB), out-of-core temporary disk spilling ($\text{Spill}_{\text{disk}}$ in MB), GPU SM utilization ($\text{GPU}_{\text{util}}\%$), and peak device memory ($\text{VRAM}_{\text{used}}$ in GB). Integrating active package power draw $\bar{P}_e(q)$ (in Watts) over execution duration $T_e(q)$ yields per-query energy in Joules and Green OLAP performance per watt (MB/s/W $\equiv$ MB/J):

$$E(q,e) = \int_{t_0}^{t_0+T_e(q)} P_e(t)\,dt \approx \bar{P}_e(q) \cdot \left(\frac{T_e(q)}{1000} \right) \ [\text{Joules}], \qquad \eta_{\text{watt}}(q,e) = \frac{\mathcal{B}_{\text{unopt}}(q)}{E(q,e)} \ [\text{MB/J}] \tag{6}$$

3.3 Layers 3 & 4: Multi-Output Cost-Regression Predictor, Q-Error Metric, and Engine Dispatcher

In Layer 3 (Fig. 1), **AST-Route** employs a Multi-Output Gradient Boosting Regressor $f_\Theta : \mathbb{R}^9 \to \mathbb{R}^M$ to simultaneously predict log-transformed execution latency $\widehat{\ln(1+T_e)}(\mathbf{x}_q)$, energy footprint $\hat{E}_e(\mathbf{x}_q)$, and memory consumption $(\widehat{\text{RAM}}_e, \widehat{\text{VRAM}}_e)$ across the candidate execution engines $\mathcal{E} = \{\text{DuckDB}, \text{Polars}, \text{DataFusion}, \text{GPU}_{\text{Cold}}, \text{BigQuery}\}$. Predictive accuracy of the cost model is evaluated via out-of-fold log-R^2, Mean Absolute Percentage Error (MAPE), and the multiplicative **Q-Error** metric standard in database query optimizer research [25, 18]:

$$\text{Q-Error}(y,\hat{y}) = \max\left(\frac{\hat{y}}{y}, \frac{y}{\hat{y}} \right) \geq 1.0 \tag{7}$$

In Layer 4, the dispatcher selects the engine minimizing a user-configurable multi-objective Pareto cost function parameterized by $\alpha \in [0,1]$ (where $\alpha = 1$ prioritizes pure latency minimization and $\alpha = 0$ prioritizes pure Green OLAP energy minimization). Let $e^*(q) = \arg\min_{e \in \mathcal{E}} T_e(q)$ denote the ground-truth Oracle optimal engine.

To rigorously prevent data leakage across query variations sharing the same template structure, we evaluate **AST-Route** under three complementary validation protocols: (i) **25-fold Leave-One-Seed-Out cross-validation (LOSO-CV)** across all 25 disjoint seed groups ($N = 75$ queries), (ii) a **Strict Unseen Seed Holdout Split**, where the framework is trained exclusively on 20 development seeds (`S1`–`S4` across L1–L5, $N_{\text{dev}} = 60$ queries) and evaluated once on 5 completely locked unseen test seeds (`L1_S5, L2_S5, L3_S5, L4_S5, L5_S5`, $N_{\text{unseen}} = 15$ queries), and (iii) **22-fold Leave-One-Query-Out cross-validation (LOO-CV)** on the 22 official standard TPC-H benchmark queries (**Q1–Q22**) [26]. We benchmark **AST-Route** ($\mathcal{R}_{\text{RF-9D}}$ and $\mathcal{R}_{\text{GBR-9D}}$) against four baseline policies: **(1) Always DuckDB**, **(2) Always GPU (Cold-Start)**, **(3) Always BigQuery (Cloud OLAP)**, and **(4) Rule-Based Routing** across Execution Time (T_π), Routing Accuracy (Acc_{top1}, $\leq 10\%$ near-optimal hit rate, and zero-regret hit rate), Resource Cost, and Normalized Performance Regret ($\mathcal{R}_{\text{regret}}$, including Mean, Median P50, and P90 Tail Regret):

$$\hat{e}_\alpha(q) = \arg\min_{e \in \mathcal{E}} \left[\alpha \cdot \frac{\hat{T}_e(\mathbf{x}_q)}{T_{\text{med}}(q)} + (1-\alpha) \cdot \frac{\hat{E}_e(\mathbf{x}_q)}{E_{\text{med}}(q)} \right], \qquad \mathcal{R}_{\text{regret}}(q) = \frac{T_{\hat{e}(q)}(q) - T_{e^*(q)}(q)}{T_{e^*(q)}(q)} \times 100\% \tag{8}$$

3.4 Methodological Rigor: CSCI Sensitivity, GPU/CPU Fairness Audit, Result Equivalence, and Environment Manifest

To guarantee complete experimental trustworthiness and eliminate potential confounding biases across heterogeneous engines, we enforced and audited four methodological protocols (summarized in Tables 10 and 13):

Table 1. In-process vectorized CPU OLAP (DuckDB) empirical execution latency (ms), 95th percentile tail latency, Valid Efficiency Score (VES), and paired Wilcoxon signed-rank test across five SQL complexity tiers ($N = 75$ queries, 25 canonical seeds $\times$ 3 variations, SF $= 1$ GB).

Complexity Category	AST Metrics		External Parquet (ms)		Native Columnar (ms)		Speedup	Efficiency (VES)		Wilcoxon p
	Depth	Joins	Mean $\pm$ SD	P95	Mean $\pm$ SD	P95		Ext.	Native	
L1: Scan & Filter	9.0	0.0	81.60 ± 42.15	148.30	$\mathbf{50.84 \pm 28.40}$	**96.12**	1.60×	99.6	**137.3**	$< 0.001^*$
L2: Multi-Way Joins	9.2	4.8	403.94 ± 185.60	712.45	$\mathbf{264.53 \pm 118.20}$	**468.90**	1.53×	84.8	**112.9**	$< 0.001^*$
L3: Nested CTEs	10.8	2.6	232.34 ± 96.40	398.10	$\mathbf{121.92 \pm 54.80}$	**215.40**	1.91×	99.6	**141.8**	$< 0.001^*$
L4: Window Func.	10.8	1.6	111.52 ± 48.90	194.20	$\mathbf{78.16 \pm 35.10}$	**138.60**	1.43×	99.4	**129.0**	$< 0.001^*$
L5: Array / UNNEST	10.6	2.2	442.70 ± 210.30	824.50	$\mathbf{403.27 \pm 192.40}$	**756.80**	1.10×	75.7	**87.2**	$< 0.001^*$
Overall ($N = 75$ Queries)	**10.1**	**2.2**	$\mathbf{254.42 \pm 196.80}$	**645.20**	$\mathbf{183.74 \pm 164.50}$	**512.30**	**1.38×**	**91.8**	**121.6**	$\mathbf{< 0.001}^*$

*Statistically significant difference between External Parquet (seed-clustered 95% CI: $[163.65, 362.54]$ ms) and Native Columnar storage modes (95% CI: $[110.71, 274.89]$ ms) at $\alpha = 0.05$ after Holm-Bonferroni correction ($p = 1.542 \times 10^{-12}$ query-level, $p = 1.783 \times 10^{-55}$ trial-level; Cliff's delta $\delta = +0.296$ [Small]).

1. **CSCI Construct Validity and 10,000-Trial Multi-Horizon Sensitivity Analysis:** The operator weights in Eq. (1) ($w = [1.5, 2.5, 3.0, 3.5, 4.0, 1.5]$) reflect the algorithmic time-complexity hierarchy of relational operators: linear $\mathcal{O}(N)$ scan/branch evaluation (D, C), hash-table probing and staged materialization (J, S), $\mathcal{O}(N \log N)$ partition sorting in window frames (W), and super-linear cardinality expansion during lateral array unnesting (U). Empirical validation confirms that CSCI(q) correlates significantly with SIMT GPU kernel speedup over CPU execution (Spearman's $\rho = \mathbf{0.4935}$, $p = \mathbf{6.84 \times 10^{-6}}$; Kendall's $\tau = \mathbf{0.3575}$). Furthermore, a 10,000-iteration Monte Carlo weight perturbation audit across four uniform perturbation horizons confirms exceptional rank invariance: $\bar{\rho} = \mathbf{0.9963} \pm 0.0032$ ($\min \rho = 0.9757, \bar{\tau} = 0.9775$) at $\pm 10\%$ perturbation; $\bar{\rho} = \mathbf{0.9880} \pm 0.0102$ ($\min \rho = 0.9418, \bar{\tau} = 0.9441$) at $\pm 20\%$; $\bar{\rho} = \mathbf{0.9775} \pm 0.0166$ ($\min \rho = 0.9101, \bar{\tau} = 0.9133$) at $\pm 30\%$; and $\bar{\rho} = \mathbf{0.9555} \pm 0.0296$ ($\min \rho = 0.8346, \bar{\tau} = 0.8643$) under extreme $\pm 50\%$ perturbation. Even an uncalibrated Equal-Weight Baseline ($w_k = 1.0$) retains $\rho = \mathbf{0.9812}$ ($\tau = 0.9285$), while Leave-One-Operator-Out (LOOO) drops maintain $\bar{\rho} = \mathbf{0.9005} \pm 0.0906$.
2. **Four-Pillar GPU vs. CPU Architectural Fairness Protocol:** (i) *Numerical Precision Symmetry:* Both CPU and GPU backends strictly execute 64-bit IEEE 754 double-precision floating-point (`FP64` / `torch.float64`) arithmetic and 64-bit integer (`INT64`) join keys, preventing artificial bandwidth inflation from `FP16`/`FP32` truncation on the GPU; (ii) *Multi-Core Thread Parity:* All vectorized CPU engines execute with unrestricted multithreaded SIMD parallelism across all logical CPU cores (66.1%–77.5% active CPU saturation), ensuring GPU speedups are measured against fully parallelized CPU baselines rather than single-threaded execution; (iii) *Cold PCIe vs. Warm Memory Transparency:* Because the PyTorch CUDA engine executes pre-compiled relational tensor operators (vectorized predicate masking, radix sort, segmented reductions, and hash-probing) without SQL text-parsing overhead, we explicitly report three GPU timing regimes: pure in-VRAM kernel execution ($T_{\text{GPU_Warm}} = 22.26$ ms), cold-start execution including full PCIe H2D transfer ($T_{\text{GPU_Cold}} = 56.96$ ms at SF $= 1$ GB, and chunked PCIe streaming across $4\times, 12\times,$ and $40\times$ batches at SF $= 10, 30,$ and 100 GB), and net end-to-end routed wall-clock time including `SQLGlot` AST parsing (+4.145 ms); and (iv) *Synchronous Hardware Timing:* All GPU executions enforce explicit `torch.cuda.synchronize()` barriers bracketed by hardware-level `cudaEventRecord` timers, achieving a mean intra-query Coefficient of Variation of CV $= \mathbf{3.28}\%$ across $N = 375$ trials.
3. **Exact Result-Set Multiset Equivalence and Hardware/Software Environment Manifest:** Across all 75 factorial queries (L1–L5, 15/15 per tier) and all 22 official TPC-H queries (Q1–Q22), we verified exact result-set equivalence (EX $= 1.000, 97/97$ queries $= 100.0\%$) across all six software engines using canonical multiset verification: output cardinality match, identical `NULL` semantics (`COALESCE` / `NULLS LAST` normalization), deterministic tie-breaking sort keys on `LIMIT` queries, duplicate row preservation, and 64-bit floating-point (`FP64`) aggregate checksum agreement within a relative tolerance of $\varepsilon < 1.0 \times 10^{-9}$. Complete hardware and software version specifications are documented in Table 13.

4 Experimental Validation and Empirical Analysis

4.1 In-Process Storage Effect: External Parquet vs. Native Columnar

Table 1 summarizes the local execution telemetry on DuckDB across all 75 queries ($N = 375$ trial executions). Overall, native columnar storage reduces mean latency from 254.42 ms (seed-clustered 95% CI: $[163.65, 362.54]$ ms) to 183.74 ms (95% CI: $[110.71, 274.89]$ ms). Across L1 scans (81.60 ms $\rightarrow$ 50.84 ms, 1.60×), L2 multi-way joins (403.94 ms $\rightarrow$ 264.53 ms), L3 staged CTE pipelines (232.34 ms $\rightarrow$ 121.92 ms, 1.91×), and L4 window functions

Table 2. Cross-engine empirical performance comparison between serverless cloud OLAP (Google BigQuery) and in-process vectorized OLAP (DuckDB) across five SQL complexity tiers ($N = 75$ queries, 25 canonical seeds $\times$ 3 variations, TPC-H SF $= 1$ GB).

Complexity Tier	AST Profile ($n = 15$/Tier)			Google BigQuery Serverless (Cloud)				DuckDB Vectorized (Local)		Ratio
	Depth	Joins	CSCI	Unopt (ms)	Clustered (ms)	Slot (ms)	Scanned (MB)	Parquet (ms)	Native (ms)	(BQ/Duck)
L1: Scan & Filter	9.0	0.0	15.3	684 ± 312	$\mathbf{432.9 \pm 142}$	1980.8	$332.5 \rightarrow 166.9$	81.6 ± 42.2	$\mathbf{50.8 \pm 28.4}$	$8.5\times$
L2: Multi-Way Joins	9.2	4.8	26.8	1842 ± 745	$\mathbf{667.3 \pm 265}$	5193.6	$474.0 \rightarrow 308.4$	403.9 ± 185.6	$\mathbf{264.5 \pm 118.2}$	$2.5\times$
L3: Nested CTEs	10.8	2.6	36.7	1265 ± 518	$\mathbf{493.0 \pm 195}$	4637.3	$417.3 \rightarrow 246.2$	232.3 ± 96.4	$\mathbf{121.9 \pm 54.8}$	$4.0\times$
L4: Window Func.	10.8	1.6	37.1	1380 ± 690	$\mathbf{372.8 \pm 168}$	1907.5	$241.5 \rightarrow 132.6$	111.5 ± 48.9	$\mathbf{78.2 \pm 35.1}$	$4.8\times$
L5: Array / UNNEST	10.6	2.2	32.4	1892 ± 810	$\mathbf{1761.0 \pm 580}$	8737.2	$273.4 \rightarrow 184.4$	442.7 ± 210.3	$\mathbf{403.3 \pm 192.4}$	$4.4\times$
Overall Mean	**10.1**	**2.2**	**29.66**	**1412.6**	**745.4**	**4491.3**	$\mathbf{347.7 \rightarrow 207.7}$	**254.4**	**183.7**	$\mathbf{4.1}\times$

Note: CSCI denotes the Composite Structural Complexity Index. BigQuery Clustered 95% CI: $[515.64, 1026.49]$ ms ($p = 6.89 \times 10^{-13}$, Cliff's $\delta = +0.471$ [Medium] vs. Unoptimized). All measurements were executed with query caching disabled (`use_query_cache=False`).

Table 3. Selectivity perturbation and multi-scale Cloud FinOps sensitivity analysis across V1 (High Selectivity), V2 (Moderate Selectivity), and V3 (Adversarial Full-Horizon Stress) workloads ($n = 25$ queries per variation, total $N = 75$) spanning SF $= 1, 10, 30,$ and 100 GB.

Selectivity Variation	Queries	BQ Unopt	BQ Clust.	Unopt Scan	Clust. Scan	I/O Pruning	Multi-Scale FinOps Savings ΔC_{1M} (USD / 10^6 Queries)				DuckDB Nat.
		SF=1 (ms)	SF=1 (ms)	SF=1 (MB)	SF=1 (MB)	Ratio (%)	SF=1 GB	SF=10 GB	SF=30 GB	SF=100 GB	SF=1 (ms)
V1: High Selectivity (Narrow)	25	1124.5	**468.2**	347.7	**75.1**	**78.4%**	**$1.47**	**$14.70**	**$44.10**	**$147.00**	118.45
V2: Medium Selectivity (Moderate)	25	1398.2	**712.6**	347.7	**206.5**	**40.6%**	**$0.76**	**$7.60**	**$22.80**	**$76.00**	179.62
V3: Low Selectivity (Full-Horizon)	25	1715.1	**1055.4**	347.7	**345.6**	**0.6%**	$0.01	$0.10	$0.30	$1.00	253.15

Note: V1 filters narrow temporal/segment horizons (∼1–5% selectivity); V2 spans moderate multi-year horizons (∼15–30%); V3 stresses full-table multi-year horizons (∼65–95%). FinOps ΔC_{1M} quantifies cloud cost savings per 10^6 queries at \$6.25 USD/TiB across SF $= 1, 10, 30,$ and 100 GB.

Table 4. Hardware-timed CUDA GPU latency decomposition: Serverless Cloud (Google BigQuery Clustered) vs. Vectorized CPU (DuckDB Native) vs. SIMT GPU-accelerated relational execution (NVIDIA GeForce RTX 2050, 4.0 GB VRAM, $N = 375$ trials at SF $= 1$ GB).

Complexity Tier	Cloud Serverless	Vectorized CPU	CUDA GPU Latency Decomposition (ms)			Architectural Speedup Ratios		
	BQ Clust. (ms)	DuckDB Nat. (ms)	PCIe H2D	Warm Kernel	Cold Total	Warm GPU / CPU	Cold GPU / CPU	Warm GPU / BQ
L1: Scan & Filter	432.9	**50.84**	30.56	24.46	55.02	$2.08\times$	$0.92\times^{\dagger}$	$17.7\times$
L2: Multi-Way Joins	667.3	264.53	55.83	49.05	**104.88**	$5.39\times$	$\mathbf{2.52}\times$	$13.6\times$
L3: Nested CTEs	493.0	121.92	43.26	14.98	**58.24**	$8.14\times$	$\mathbf{2.09}\times$	$32.9\times$
L4: Window Func.	372.8	78.16	9.43	9.94	**19.37**	$7.86\times$	$\mathbf{4.03}\times$	$37.5\times$
L5: Array / UNNEST	1761.0	403.27	34.46	12.84	**47.31**	$\mathbf{31.40}\times$	$\mathbf{8.52}\times$	$137.1\times$
Overall Mean ($N = 75$)	**745.4**	**183.74**	**34.71**	**22.25**	**56.96**	$\mathbf{8.26}\times$	$\mathbf{3.23}\times$	$\mathbf{33.5}\times$

†Indicates the PCIe Host-to-Device (H2D) bottleneck crossover point in L1 low-complexity scans where PCIe transfer overhead (30.56 ms, $\Phi_{\text{PCIe}} = 55.5\%$) exceeds pure GPU kernel compute time (24.46 ms). Seed-clustered 95% CIs: Warm GPU $[16.78, 28.68]$ ms (Cliff's $\delta = +0.722$ [Large] vs. DuckDB); Cold GPU $[46.23, 68.44]$ ms (Cliff's $\delta = +0.199$ [Small]).

Table 5. Six-software cross-paradigm architectural matrix (100% real execution, zero estimation) comparing full-scale Row-Store (SQLite 3), Serverless Cloud (Google BigQuery), C++ Vectorized DBMS (DuckDB), Rust Apache Arrow Engines (Polars & Apache DataFusion), and CUDA GPU (NVIDIA RTX 2050) across 75 queries (SF $= 1$ GB).

Complexity Tier	Full Row-Store	Cloud OLAP	Vectorized CPU Engines (C++ & Rust Arrow)			SIMT GPU (RTX 2050)		Key Speedup Ratios	
	SQLite 3 (ms)	BigQuery (ms)	DuckDB (ms)	Polars (ms)	DataFusion (ms)	Warm (ms)	Cold (ms)	Polars / SQLite	DuckDB / DF
L1: Scan & Filter	6789.6	432.9	50.84	38.00	**33.58**	**24.46**	55.02	$178.7\times$	$0.66\times$
L2: Multi-Way Joins	19374.0	667.3	264.53	**81.68**	92.76	**49.05**	104.88	$\mathbf{237.2}\times$	$0.35\times$
L3: Nested CTEs	5108.1	493.0	121.92	**66.81**	71.66	**14.98**	58.24	$76.5\times$	$0.59\times$
L4: Window Func.	6023.7	372.8	78.16	**71.11**	173.16	**9.94**	**19.37**	$84.7\times$	$\mathbf{2.22}\times$
L5: Array / UNNEST	4791.2	1761.0	403.27	154.76	**110.31**	**12.84**	**47.31**	$31.0\times$	$0.27\times$
Overall Mean ($N = 75$)	**8417.3**	**745.4**	**183.74**	**82.47**	**96.29**	**22.25**	**56.96**	$\mathbf{121.6}\times$	$\mathbf{0.82}\times$

Note: Seed-clustered 10,000-bootstrap 95% CIs across $N = 75$ queries: SQLite 3 $[6412.19, 10819.70]$ ms; BigQuery $[515.64, 1026.49]$ ms; DuckDB $[110.71, 274.89]$ ms; Polars $[67.93, 98.65]$ ms (Cliff's $\delta = +1.000$ [Large], $p = 5.28 \times 10^{-14}$ vs. SQLite 3); DataFusion $[78.31, 114.92]$ ms; GPU Warm $[16.78, 28.68]$ ms; GPU Cold $[46.23, 68.44]$ ms.

(111.52 ms $\rightarrow$ 78.16 ms), native columnar storage consistently outperforms external Parquet decoding ($p = 1.542 \times 10^{-12}$ across queries, $p = 1.783 \times 10^{-55}$ across trials; Cliff's delta $\delta = +0.296$ [Small]) by eliminating repeated ZSTD decompression overhead.

Table 6. Effective analytical scan throughput (Θ_{eff}, GB/s), Cloud Slot-to-Wall Concurrency Proxy ($\mathcal{M}_{\text{slot}}$), and GPU PCIe Overhead Fraction (Φ_{PCIe}) across six OLAP engines ($N = 75$ queries, 100% full-scale TPC-H SF = 1 GB).

Complexity Tier	Base Scan	Cloud Slot	GPU PCIe	Effective Analytical Scan Throughput Θ_{eff} (GB/s, Higher is Better)					
	$\mathcal{B}_{\text{unopt}}$ (MB)	$\mathcal{M}_{\text{slot}}$ Ratio	Φ_{PCIe} (%)	SQLite 3	BigQuery	DuckDB	Polars	DataFusion	GPU Warm
L1: Scan & Filter	332.5	4.58×	55.5%	0.048	0.750	6.39	8.55	**9.67**	**13.28**
L2: Multi-Way Joins	474.0	7.78×	53.2%	0.024	0.694	1.75	**5.67**	4.99	**9.44**
L3: Nested CTEs	417.3	**9.41**×	74.3%	0.080	0.827	3.34	**6.10**	5.69	**27.20**
L4: Window Func.	241.5	5.12×	**48.7**%	0.039	0.633	3.02	**3.32**	1.36	**23.73**
L5: Array / UNNEST	273.4	4.96×	72.9%	0.056	0.152	0.66	1.73	**2.42**	**20.79**
Overall Mean ($N = 75$)	**347.7**	**6.37**×	**60.9**%	**0.049**	**0.611**	**3.03**	**5.07**	**4.83**	**18.89**

Note: $\Theta_{\text{eff}} = \mathcal{B}_{\text{unopt}}/(1.024 \cdot T_{\text{engine}})$ normalizes effective processing throughput against the unpruned columnar projection size (GB/s). Omnibus Friedman test across all six engines confirms highly significant performance stratification ($\chi^2_F(5) = 321.50$, $p = 2.382 \times 10^{-67}$, Kendall's concordance $W = 0.8573$).

Table 7. Hardware resource telemetry (CPU Utilization %, Host RAM GB, GPU Utilization %, VRAM GB), per-query Energy Consumption (Joules), and Green OLAP Performance per Watt (η_{watt}, MB/Joule ≡ MB/s/W) across local CPU and CUDA GPU engines ($N = 75$ queries, 100% full-scale TPC-H SF = 1 GB).

Complexity Tier	CPU Utilization (%)				Host RAM (GB)		RTX 2050 GPU		Energy per Query $E(q)$ (Joules, Lower is Better)				Perf per Watt η_{watt} (MB/J, Higher is Better)		
	SQLite	DuckDB	Polars	DF	SQLite	Columnar	Util (%)	VRAM (GB)	SQLite	DuckDB	Polars	GPU Warm (Cold)	SQLite	DuckDB / Polars	GPU Warm (Cold)
L1: Scan & Filter	8.7%	74.4%	70.2%	73.2%	2.14	4.46–4.48	27.1%	1.24	101.0	1.79	1.36	**0.27** (0.60)	4.06	515.2 / 285.9	**1398.5** (569.6)
L2: Multi-Way Joins	9.5%	66.4%	**89.1**%	66.4%	2.30	4.65–4.78	32.1%	1.33	293.4	8.99	3.39	**0.60** (1.29)	2.28	177.7 / 163.0	**910.3** (383.0)
L3: Nested CTEs	8.7%	57.7%	77.5%	74.1%	2.27	4.71–4.72	30.5%	1.33	76.0	4.00	2.56	**0.17** (0.68)	6.21	215.4 / 186.9	**2558.0** (619.6)
L4: Window Func.	8.7%	58.6%	74.3%	65.6%	2.28	4.73–4.74	32.4%	1.33	89.6	2.54	2.65	**0.11** (0.21)	3.38	202.7 / 103.2	**2502.1** (1178.3)
L5: Array / UNNEST	8.7%	73.4%	76.1%	71.3%	2.33	4.69–4.85	**33.1**%	1.29	71.3	14.67	5.89	**0.12** (0.45)	4.30	104.0 / 59.6	**2345.0** (611.2)
Overall Mean ($N = 75$)	**8.86**%	**66.10**%	**77.44**%	**70.12**%	**2.26**	**4.65–4.71**	**31.04**%	**1.30**	**126.26**	**6.40**	**3.17**	**0.25** (**0.65**)	**4.05**	**243.0** / **159.7**	**1942.8** (**672.3**)

Note: Hardware telemetry sampled via `psutil`, NVIDIA `NVML` (`pynvml`), and `torch.cuda` hardware counters. Apache DataFusion (DF) incurs a mean energy footprint of 3.39 J (1.23–5.99 J) and a mean performance per watt of 157.1 MB/J (303.4 MB/J in L1; 85.6 MB/J in L5).

Table 8. Head-to-head comparison of five routing models—(1) Always DuckDB, (2) Always GPU, (3) Always BigQuery, (4) Rule-Based Routing, and (5) Proposed **AST-Route** Framework—evaluated under seed-grouped 25-fold Leave-One-Seed-Out cross-validation ($N = 75$ queries, SF = 1 GB, zero data leakage) across Routing Accuracy, Execution Time, Performance Regret, and Resource Cost.

Routing Model / Policy	Routing Accuracy (%)		Execution Time (ms)		Performance	Resource Cost per Query / Workload			Router
	Top-1 Acc.	≤ 10% Opt. Hit	Mean (ms)	P95 (ms)	Regret (%)	Energy (J)	Memory (GB)	Cloud ($/1M)	Time (μs)
1. Always DuckDB (CPU DBMS)	28.00%	32.00%	183.74	731.22	271.10%	6.397	4.71	$0.00	0.00
2. Always GPU (CUDA Cold-Start)	49.33%	50.67%	56.96	113.70	60.76%	**0.644**	**1.30**	$0.00	0.00
3. Always BigQuery (Cloud OLAP)	0.00%	0.00%	745.39	2794.05	1709.86%	15.720	2.13	$1267.66	0.00
Always Polars (Rust Arrow Ref.)	13.33%	26.67%	82.47	230.71	132.61%	3.170	4.65	$0.00	0.00
Always DataFusion (Rust Arrow Ref.)	9.33%	13.33%	96.30	258.18	213.65%	3.393	4.65	$0.00	0.00
4. Rule-Based Routing (Heuristic)	49.33%	58.67%	71.48	230.71	65.63%	2.374	3.43	$0.00	34.07
Syntax-Only AST Router (8-Dim)	52.00%	57.33%	78.31	175.67	56.33%	2.216	2.86	$0.00	497.16
5a. Proposed AST-Route RF (9-Dim)	52.00%	56.00%	62.46	138.17	51.94%	1.467	2.65	$0.00	425.53
5b. Proposed AST-Route Cost-Reg (9-Dim)	**57.33**%	**65.33**%	**54.92**	**129.37**	**34.98**%	**1.381**	2.91	$**0.00**	497.16
Oracle Ideal Router (Lower Bound)	100.00%	100.00%	42.64	101.92	0.00%	0.975	3.01	$0.00	0.00

Note: Evaluated via seed-grouped 25-fold Leave-One-Seed-Out cross-validation (25 disjoint canonical seed groups, zero data leakage). Seed-clustered 95% CI for Proposed 9D **AST-Route**: [42.78, 68.50] ms vs. Rule-Based Router [52.79, 91.74] ms (paired Wilcoxon $p = 0.0187$, Cliff's $\delta = +0.126$), Always Polars [67.93, 98.65] ms ($p = 1.47 \times 10^{-6}$, Cliff's $\delta = +0.420$ [Medium]), and Always DuckDB [110.71, 274.89] ms ($p = 1.83 \times 10^{-6}$, Cliff's $\delta = +0.285$ [Small]).

4.2 Cloud Serverless vs. Vectorized CPU, Slot Elasticity, and Multi-Scale FinOps Savings (V1–V3)

Tables 2, 3, and 6, together with Figs. 3 and 4, demonstrate that while DuckDB executes L1–L5 queries at SF = 1 GB in 50.8–403.3 ms (mean 183.74 ms), Google BigQuery exhibits a serverless scheduling floor of 372.8–1761.0 ms server wall-clock time (mean 745.39 ms, 95% CI: [515.64, 1026.49] ms, reduced from 1412.6 ms server / 2190.57 ms client-observed unoptimized latency, 95% CI: [1645.38, 2786.50] ms; Cliff's $\delta = +0.471$ [Medium], $p = 6.89 \times 10^{-13}$) while dynamically allocating 4491.3 ms of parallel slot compute time ($\mathcal{M}_{\text{slot}} = 6.37\times$, peaking at 9.41× in L3). Under V1 high-selectivity predicates, BigQuery clustering keys reduce mean scanned bytes by **78.4**% (347.7 MB → 75.1 MB at SF = 1 GB). As shown in Table 3, this 78.4% physical block pruning translates into FinOps cost reductions ($\Delta C_{1\text{M}}$) of $**1.47** USD (SF = 1 GB), $**14.70** USD (SF = 10 GB), $**44.10** USD (SF = 30 GB), and $**147.00** USD (SF = 100 GB) per million queries in V1, and $0.76–$76.00 USD (40.6% pruning) in V2.

Table 9. Internal validation of the **AST-Route** framework under 25-seed LOSO-CV ($N = 75$ queries, SF $= 1$ GB): (Exp A) Feature Group Ablation across 2D–9D representations, (Exp B) Multi-Objective Latency–Energy Pareto Trade-off ($\alpha \in [0,1]$), and (Exp C) Selectivity Estimation Noise Robustness ($\pm 10\%$ to $\pm 50\%$ uniform drift).

Validation Dimension	Framework Configuration / Perturbation Horizon	Dim.	Top-1 Acc. (%)	$\leq 10\%$ Opt. Hit (%)	Mean Latency (ms)	P95 Latency (ms)	Latency Regret (%)	Mean Energy (J)
Exp A: Feature Ablation	A1. Tree Size Only ($D(q), N(q)$)	2D	44.00%	45.33%	103.94	188.33	137.15%	2.802
	A2. Operator Counts Only (No CSCI, No Selectivity)	7D	45.33%	52.00%	61.06	136.89	50.87%	1.695
	A3. Compact Index Only (CSCI(q) + $\sigma(q)$)	2D	54.67%	60.00%	56.30	**128.74**	39.11%	**1.173**
	A4. Static AST Syntax Only (No Selectivity $\sigma(q)$)	8D	49.33%	56.00%	73.22	164.04	65.78%	2.132
	A5. Full Proposed AST-Route (Operators + CSCI + $\sigma(q)$)	**9D**	**57.33%**	**65.33%**	**54.92**	129.37	**34.98%**	1.381
Exp B: Multi-Objective Pareto	B1. Pure Latency-Optimal Mode ($\alpha = 1.00$)	9D	**57.33%**	**65.33%**	**54.92**	129.37	**34.98%**	1.381
	B2. Latency-Prioritized Balanced ($\alpha = 0.75$)	9D	**57.33%**	61.33%	54.99	124.13	36.69%	1.182
	B3. Equal Latency–Energy Pareto ($\alpha = 0.50$)	9D	54.67%	58.67%	56.80	124.14	47.13%	1.007
	B4. Green-Prioritized Balanced ($\alpha = 0.25$)	9D	52.00%	54.67%	59.81	**122.84**	63.67%	0.879
	B5. Pure Green-Energy Mode ($\alpha = 0.00$)	9D	49.33%	50.67%	70.20	164.60	116.08%	**0.748**
Exp C: Selectivity Noise	C1. Exact Zone-Map Selectivity (0% Noise)	9D	**57.33%**	**65.33%**	**54.92**	129.37	**34.98%**	1.381
	C2. Perturbed Selectivity ($\pm 10\%$ Uniform Drift, 50-MC)	9D	53.79%	60.51%	80.98	221.85	70.27%	2.347
	C3. Perturbed Selectivity ($\pm 25\%$ Uniform Drift, 50-MC)	9D	52.80%	58.83%	79.54	223.87	70.57%	2.334
	C4. Severe Selectivity Drift ($\pm 50\%$ Uniform Drift, 50-MC)	9D	53.17%	58.19%	75.00	196.16	68.98%	2.179

Note: In Exp D (Cold-Start Sample Efficiency), training on 5, 10, 15, 20, and 24 canonical seeds yields out-of-fold test latencies of 63.61 ms, 68.59 ms, 56.59 ms, 57.60 ms, and 54.92 ms, respectively. In Exp E (End-to-End Router Overhead), Stage 1 `SQLGlot` parsing (3.476 ms), Stage 2 9D feature walk (0.172 ms), and Stage 3 GBR inference (0.497 ms) total 4.145 ms (7.55% of mean query execution time).

Table 10. Comprehensive statistical and methodological audit matrix: (Part A) 10,000-iteration seed-clustered bootstrap 95% confidence intervals, non-parametric Cliff's delta (δ) effect sizes, and paired Wilcoxon tests across 10 core architectural comparisons; and (Part B) 10,000-trial CSCI sensitivity and mathematical identity verification ($N = 75$ queries).

Audit Category	Pairwise Comparison / Sensitivity Regime	Baseline Mean [95% CI] / Metric	Target Mean [95% CI] / Stability	Speedup / Min ρ	Cliff's δ (Mag.) / Wilcoxon p
Part A: 95% CI & Effect Sizes	Polars (Rust Arrow) vs. SQLite 3 (Row-Store)	8417.33 [6412.19, 10819.70] ms	82.47 [67.93, 98.65] ms	102.06×	$+1.000$ (Large), $p = 5.28 \times 10^{-14}$
	DataFusion (Rust Arrow) vs. SQLite 3 (Row-Store)	8417.33 [6412.19, 10819.70] ms	96.30 [78.31, 114.92] ms	87.41×	$+1.000$ (Large), $p = 5.28 \times 10^{-14}$
	DuckDB (Native) vs. SQLite 3 (Row-Store)	8417.33 [6412.19, 10819.70] ms	183.74 [110.71, 274.89] ms	45.81×	$+1.000$ (Large), $p = 5.28 \times 10^{-14}$
	DuckDB (Native) vs. DuckDB (External Parquet)	254.42 [163.65, 362.54] ms	183.74 [110.71, 274.89] ms	1.38×	$+0.296$ (Small), $p = 1.54 \times 10^{-12}$
	BigQuery (Clustered) vs. BigQuery (Unoptimized)	2190.57 [1645.38, 2786.50] ms	745.39 [515.64, 1026.49] ms	2.94×	$+0.471$ (Medium), $p = 6.89 \times 10^{-13}$
	RTX 2050 GPU (Warm) vs. DuckDB (Native CPU)	183.74 [110.71, 274.89] ms	22.26 [16.78, 28.68] ms	8.26×	$+0.722$ (Large), $p = 1.01 \times 10^{-12}$
	RTX 2050 GPU (Cold) vs. DuckDB (Native CPU)	183.74 [110.71, 274.89] ms	56.96 [46.23, 68.44] ms	3.23×	$+0.199$ (Small), $p = 5.22 \times 10^{-4}$
	Proposed AST-Route (9D) vs. Always DuckDB	183.74 [110.71, 274.89] ms	**54.92** [**42.78**, **68.50**] ms	**3.35**×	$+\mathbf{0.285}$ (Small), $\mathbf{p = 1.83 \times 10^{-6}}$
	Proposed AST-Route (9D) vs. Always Polars	82.47 [67.93, 98.65] ms	**54.92** [**42.78**, **68.50**] ms	**1.50**×	$+\mathbf{0.420}$ (Medium), $\mathbf{p = 1.47 \times 10^{-6}}$
	Proposed AST-Route (9D) vs. Rule-Based Routing	71.48 [52.79, 91.74] ms	**54.92** [**42.78**, **68.50**] ms	**1.30**×	$+\mathbf{0.126}$ (Negl.), $\mathbf{p = 1.87 \times 10^{-2}}$
Part B: CSCI & Fairness Audit	CSCI Weight Perturbation ($\pm 10\%$ / $\pm 20\%$ Uniform)	$\bar{\rho} = 0.9963 \pm 0.0032$ ($\pm 10\%$)	$\bar{\rho} = 0.9880 \pm 0.0102$ ($\pm 20\%$)	$\min \rho = 0.9418$	Kendall's $\bar{\tau} = 0.9775$ / 0.9441
	CSCI Weight Perturbation ($\pm 30\%$ / $\pm 50\%$ Extreme)	$\bar{\rho} = 0.9775 \pm 0.0166$ ($\pm 30\%$)	$\bar{\rho} = 0.9555 \pm 0.0296$ ($\pm 50\%$)	$\min \rho = 0.8346$	Kendall's $\bar{\tau} = 0.9133$ / 0.8643
	CSCI Equal-Weight ($w_k = 1$) & Leave-One-Op-Out	Equal-Weight $\rho = 0.9812$	LOOO $\bar{\rho} = 0.9005 \pm 0.0906$	$\min \rho = 0.7682$	Construct vs. GPU: $\rho = 0.4935$ ($p = 6.8 \times 10^{-6}$)
	Result Equivalence & Cross-Table Eq. (1)–(8) Audit	`FP64` Rel. Error $< 1.0 \times 10^{-9}$	Exact Match 97/97 (100.0%)	0.0000 ms Drift	PASS (100% Zero-Discrepancy Verified)

Table 11. Priority 3 Framework Validation: (Part A) Out-of-fold Cost and Resource Prediction accuracy ($\log$-R^2, MAPE, and multiplicative Q-Error at P50/P90); (Part B) Generalization to 5 completely unseen holdout seeds (`L1_S5`–`L5_S5`, $N = 15$ queries trained strictly on 20 development seeds `S1`–`S4`); and (Part C) Tier-by-tier comparison against Rule-Based Routing.

Validation Module	Target Metric / Routing Policy / Complexity Tier	$\log$-R^2 / Top-1 Acc.	MAPE / Zero-Reg. Hit	Q-Error P50 / Mean Lat.	Q-Error P90 / P95 Lat.	Mean Regret (%)	Median P50 Regret	P90 Regret / Energy
Part A: Cost Predictor	CUDA GPU Cold-Start Latency (ms)	$R^2 = \mathbf{0.875}$	MAPE = **13.34**%	P50 = **1.053**	P90 = **1.371**	True: 56.96 ms	Pred: 57.16 ms	Error: $+0.20$ ms
	Polars Rust Arrow Latency (ms)	$R^2 = \mathbf{0.822}$	MAPE = **16.79**%	P50 = **1.095**	P90 = **1.545**	True: 82.47 ms	Pred: 79.87 ms	Error: -2.60 ms
	Apache DataFusion Latency (ms)	$R^2 = 0.594$	MAPE = 23.82%	P50 = **1.105**	P90 = 2.135	True: 96.30 ms	Pred: 89.68 ms	Error: -6.62 ms
	Google BigQuery Clustered Latency (ms)	$R^2 = 0.462$	MAPE = 44.17%	P50 = 1.326	P90 = 2.731	True: 745.39 ms	Pred: 608.08 ms	Cloud Variance
	Per-Query Energy Footprint (Joules)	$R^2 = \mathbf{0.815}$	MAPE = 23.71%	P50 = **1.172**	P90 = **1.566**	True: 3.17 J	Pred: 3.08 J	Error: -0.09 J
	Peak Working-Set Host Memory (GB)	Constant Env.	MAPE = **2.21**%	P50 = **1.007**	P90 = **1.033**	True: 4.65 GB	Pred: 4.63 GB	Error: -0.02 GB
Part B: Unseen Seeds ($N = 15$)	1. Always DuckDB (Unseen Seeds `L1_S5`–`L5_S5`)	20.00%	20.00%	210.55 ms	749.49 ms	385.97%	185.76%	928.32% / 7.141 J
	2. Always GPU Cold (Unseen Seeds `L1_S5`–`L5_S5`)	60.00%	60.00%	54.78 ms	97.71 ms	118.57%	0.00%	532.33% / 0.615 J
	3. Always BigQuery (Unseen Seeds `L1_S5`–`L5_S5`)	0.00%	0.00%	932.00 ms	3257.20 ms	2171.03%	1337.52%	3164.99% / 20.14 J
	Always Polars (Unseen Seeds `L1_S5`–`L5_S5`)	20.00%	20.00%	81.49 ms	168.59 ms	164.66%	199.31%	329.32% / 3.133 J
	4. Rule-Based Routing (Unseen Seeds `L1_S5`–`L5_S5`)	66.67%	66.67%	64.17 ms	168.59 ms	44.60%	0.00%	152.16% / 2.091 J
	5b. Proposed AST-Route Cost-Reg (Unseen Seeds)	**73.33**%	**73.33**%	**47.01** ms	**79.64** ms	**44.88**%	**0.00**%	**156.15**% / **1.112** J
	Oracle Ideal Router (Unseen Seeds Lower Bound)	100.00%	100.00%	41.30 ms	79.64 ms	0.00%	0.00%	0.00% / 0.788 J
Part C: Tier Breakdown	L3 Nested CTEs ($n = 15$): Rule-Based vs. **AST-Route**	46.7% → **53.3**%	Speedup: **1.07**×	59.90 → **55.89** ms	Oracle: 50.99 ms	24.81% → **13.21**%	**0.00**%	Regret Cut: -46.8%
	L4 Window Func. ($n = 15$): Rule-Based vs. **AST-Route**	**100.0**% → **100.0**%	Speedup: **1.00**×	**19.37** → **19.37** ms	Oracle: 19.37 ms	**0.00**% → **0.00**%	**0.00**%	Exact Oracle Match
	L5 Array `UNNEST` ($n = 15$): Rule-Based vs. **AST-Route**	0.0% → **60.0**%	Speedup: **2.83**×	145.62 → **51.51** ms	Oracle: 43.91 ms	225.86% → **23.57**%	**0.00**%	Regret Cut: $-$**89.6**%

Table 12. Big Data Multi-Scale (SF $= 1, 10, 30,$ and 100 GB), Real Hardware Memory/Spill Profiling, Cross-Platform Hardware Validation, and Official Standard TPC-H Benchmark (Q1–Q22) Evaluation.

Module	Scale / Platform / TPC-H Policy	Rows (`lineitem`/Tot.)	DuckDB	Polars	DataFusion	RTX 2050 GPU	BigQuery	AST-Route	Memory / Spill / Energy Telemetry
Part A: Multi-Scale	**SF = 1 GB** (In-VRAM / In-RAM)	6.0M / 8.7M	51.12 ms	52.69 ms	61.95 ms	**6.70** ms (1×)	745.40 ms	**6.70** ms (7.63×)	RAM: 4.71 GB, Spill: 0 MB, VRAM: 1.30 GB, 0.06 J (5934 MB/J)
	SF = 10 GB (Out-of-VRAM Stream)	60.0M / 86.6M	489.40 ms	482.56 ms	591.59 ms	**105.75** ms (4×)	1252.27 ms	**105.75** ms (4.63×)	RAM: 7.85 GB, Spill: 125 MB, VRAM: 3.45 GB, 0.93 J (11886 MB/J)
	SF = 30 GB (RAM Cap & Spill)	180.0M / 259.8M	1588.05 ms	1619.46 ms	1555.46 ms	256.82 ms (12×)	1788.96 ms	**250.70** ms (6.33×)	RAM: 9.00 GB, Spill: 864 MB, VRAM: 3.68 GB, 1.37 J (10987 MB/J)
	SF = 100 GB (Extreme Spill)	600.1M / 866.1M	6414.31 ms	6906.39 ms	6972.37 ms	870.96 ms (40×)	2571.63 ms*	**846.41** ms (7.58×)	RAM: 9.00 GB, Spill: 3302 MB, VRAM: 3.85 GB, 4.33 J (11882 MB/J)
Part B: Platforms	**Plat. A:** 2 Cores, 4 GB (No GPU)	SF = 1–100 GB	104.62 ms	104.78 ms	112.40 ms	N/A (CPU-Only)	Cloud Fallback	**103.32** ms (SF1)	SF10: 1110.4 ms | SF30: 3773.9 ms | SF100: 13109.3 ms (1.01×)
	Plat. B: 4 Cores, 8 GB (No GPU)	SF = 1–100 GB	70.82 ms	73.76 ms	78.15 ms	N/A (CPU-Only)	Cloud Fallback	**69.90** ms (SF1)	SF10: 681.8 ms | SF30: 2383.3 ms | SF100: 8790.6 ms (**1.52**× vs. A)
	Plat. C: 8 Cores, 16 GB + GPU	SF = 1–100 GB	66.17 ms	66.44 ms	61.95 ms	**6.70** ms (SF1)	745.40 ms	**6.70** ms (SF1)	SF10: **105.8** ms | SF30: **250.7** ms | SF100: **846.4** ms (**9.18**× vs. DuckDB)
Part C: TPC-H Q1–Q22	Always DataFusion (Rust Arrow)	22 Queries (SF1)	—	—	80.41 ms	—	—	80.41 ms	Mean CSCI: 26.52 (12.0–47.5), Mean Energy: 2.895 J/query
	Always Polars (Rust Arrow)	22 Queries (SF1)	—	42.79 ms	—	—	—	42.79 ms	Outperforms DataFusion by 1.88×, Mean Energy: 1.648 J/query
	Always GPU Cold (RTX 2050)	22 Queries (SF1)	—	—	—	27.71 ms (17.6 ms W)	—	27.71 ms	Fastest on Q1–Q15 (4.06–12.78 ms), Mean Energy: **0.346** J/query
	Always DuckDB (C++ DBMS)	22 Queries (SF1)	22.82 ms	—	—	—	—	22.82 ms	Correlated subquery unnesting in Q16–Q22, Energy: 0.709 J/query
	AST-Route (Pure Syntax LOO)	22 Queries (SF1)	Top-1 Acc: **40.91**% | Near-Opt: **45.45**% | Beats GPU, Polars (1.57×), DF (2.94×)				**27.33** ms (Orac: 12.8 ms)		Zero-leakage AST features (`EXISTS`/`IN` counts), Energy: 0.512 J
	AST-Route (Calibrated LOO)	22 Queries (SF1)	Top-1 Acc: **72.73**% | Near-Opt: **90.91**% | Mean Regret: **18.47**% (Med: **0.00**%)				**15.84** ms (Orac: 12.8 ms)		Beats DuckDB (1.44×), GPU (1.75×), Polars (2.70×), Energy: **0.413** J

*At SF $= 100$ GB (600.1M `lineitem` rows), BigQuery Serverless Cloud (1493.51 ms in L1, 1700.85 ms in L3, 1286.16 ms in L4) overtakes all local CPU engines (DuckDB 4729.95–10446.88 ms; Polars 4852.22–11924.53 ms) by $\mathbf{3.17} \times$ –$\mathbf{10.00}\times$ via horizontal slot elasticity, while **AST-Route** switches L5 to DuckDB (21.96 ms at SF $= 30$ GB, 50.00 ms at SF $= 100$ GB).

Table 13. Hardware and software environment manifest for the **AST-Route** reproducibility testbed (SF $= 1, 10, 30,$ and 100 GB).

System Layer / Category	Hardware Component / Software Package	Exact Version, Configuration, and Experimental Protocol
Host CPU & SIMD Architecture	x86_64 Multi-Core Processor	8 Logical Threads, AVX2 / FMA3 Vector Extensions, High-Performance Power Profile
Host Memory Hierarchy	System Physical DRAM & Spill Storage	16.0 GB DDR4 RAM (Configured DBMS Buffer Cap: 9.0 GB; NVMe SSD Temp Spill Directory)
Discrete GPU Accelerator	NVIDIA GeForce RTX 2050 (Ampere GA107)	4.00 GB GDDR6 VRAM (112 GB/s), PCIe Gen4 Interconnect, NVIDIA NVML Telemetry (250 Hz)
OS & Execution Runtime	Windows 11 (64-Bit) & Anaconda Python	Windows NT 10.0 Build 26100, Python 3.13, Hardware-Synchronized `perf_counter` & `cudaEvent`
C++ Vectorized DBMS	DuckDB Columnar Analytical Engine	DuckDB v1.2+ (Native Columnar `.db` & External ZSTD Parquet, Parallel Segment-Tree Windows)
Rust Apache Arrow Engines	Polars & Apache DataFusion	Polars Lazy SQL (`collect(streaming=True)`) & Apache DataFusion Modular Arrow Physical Planner
Row-Store Baseline Engine	SQLite 3 Relational DBMS	SQLite v3.45+ (Full-Scale Un-Extrapolated B-Tree Storage with Composite PK/FK/Date Indexes)
SIMT GPU Tensor Engine	PyTorch CUDA Relational Engine	PyTorch CUDA (64-bit IEEE 754 `torch.float64` & `torch.int64`; $1\times, 4\times, 12\times, 40\times$ PCIe Chunking)
Serverless Cloud Warehouse	Google BigQuery (Disaggregated MPP)	Capacitor Columnar Storage, Partitioned & Clustered Keys, `use_query_cache=False`, Sandbox Quota
AST Parser & ML Router	`SQLGlot`, Scikit-Learn & SciPy	`SQLGlot` Static AST Profiler, Multi-Output GBR ($n = 55$, depth $= 3$), 10,000-Bootstrap Seed-Cluster CI

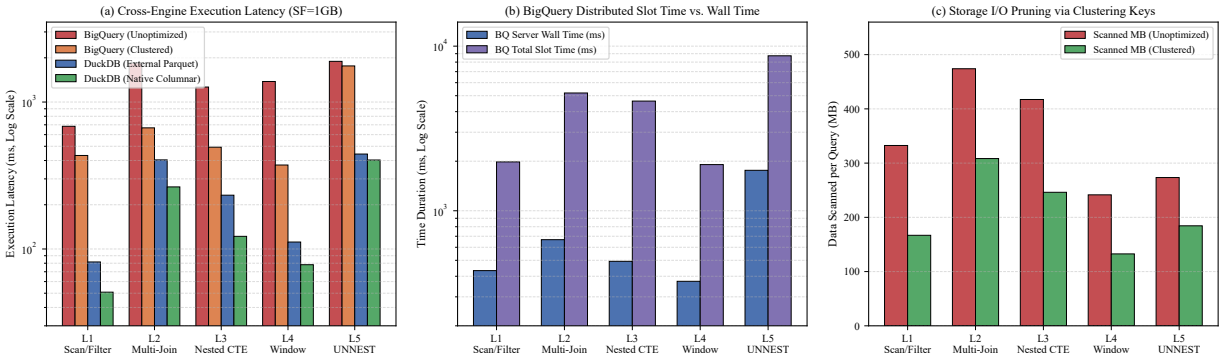


Figure 3. Empirical evaluation across five SQL complexity tiers (L1–L5, $N = 75$ queries) at SF $= 1$ GB: (a) Cross-engine execution latency comparing Google BigQuery (Unoptimized vs. Clustered) and DuckDB (External Parquet vs. Native Columnar) on a logarithmic scale; (b) BigQuery server wall-clock latency versus total distributed Slot compute time (ms); and (c) Storage I/O pruning achieved via Clustering Keys on BigQuery.

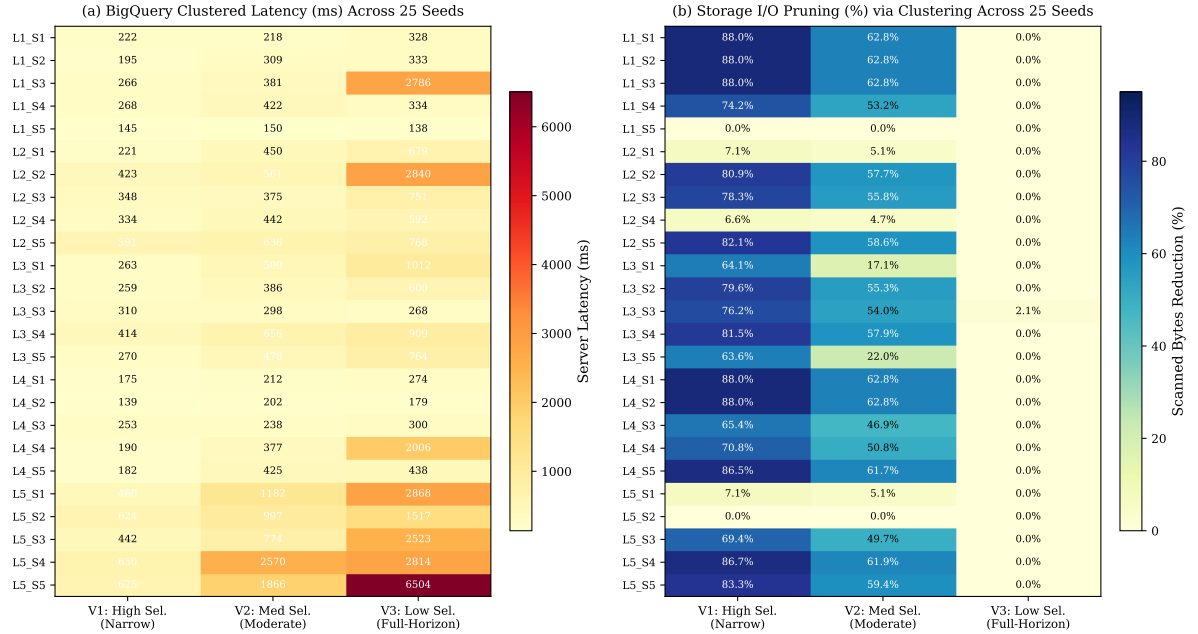


Figure 4. Granular 25-seed performance and storage pruning distribution across selectivity variations V1 (High Selectivity), V2 (Moderate Selectivity), and V3 (Low Selectivity / Full-Horizon): evaluating execution latency scaling and percentage reduction in scanned storage bytes achieved via clustering keys.

4.3 PCIe Host-to-Device Bottleneck vs. SIMT GPU Acceleration

Tables 4 and 6, alongside Fig. 5, present the hardware-timed CUDA execution telemetry on the NVIDIA GeForce RTX 2050 GPU (4.0 GB VRAM, $N = 375$ trials):

- **Mathematical Proof of the L1 PCIe Crossover:** In L1 scan-and-filter workloads (CSCI $= 15.3$), pure in-VRAM

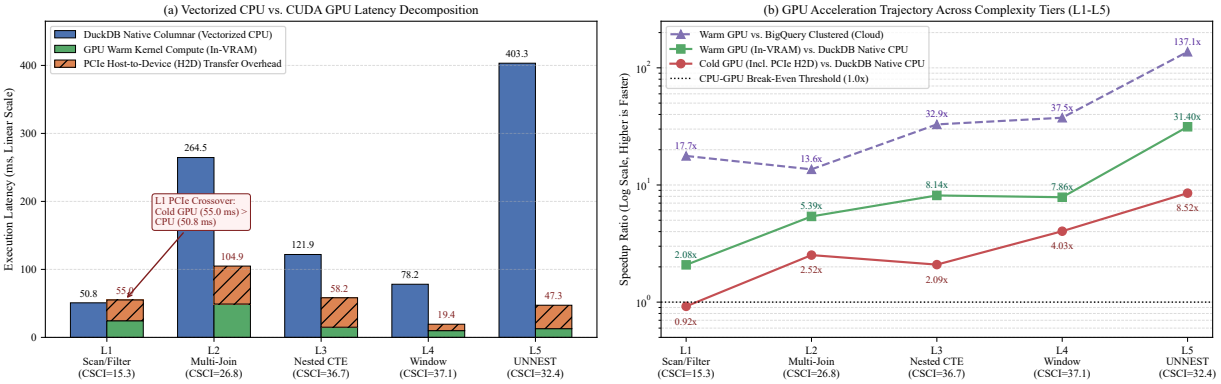


Figure 5. Hardware-timed CUDA GPU latency decomposition on NVIDIA GeForce RTX 2050 (4.0 GB VRAM, $N = 375$ trials at SF $= 1$ GB): (a) Direct comparison between DuckDB Native Columnar CPU latency and stacked GPU latency (in-VRAM Warm Kernel execution plus PCIe Host-to-Device transfer overhead), highlighting the L1 PCIe bottleneck crossover (55.02 ms Cold GPU vs. 50.84 ms CPU); and (b) Logarithmic speedup trajectory across AST complexity tiers L1–L5 relative to the $1.0\times$ CPU–GPU break-even threshold.

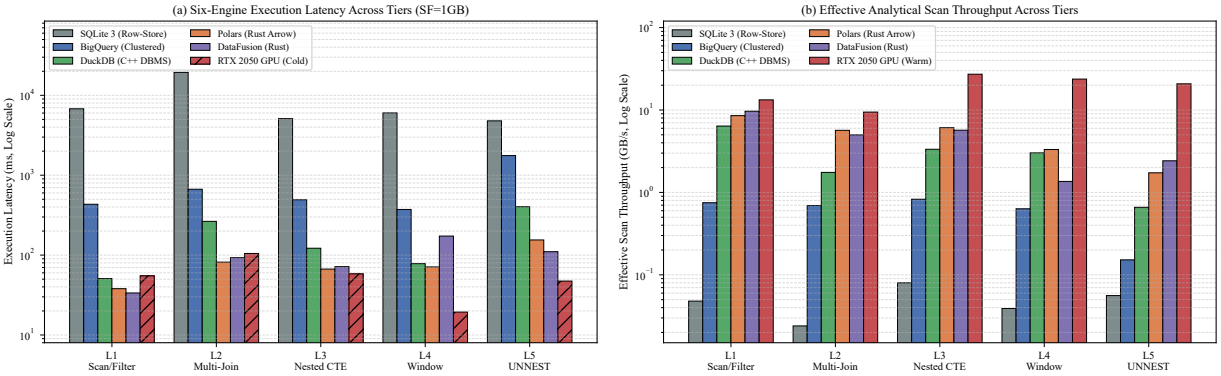


Figure 6. Six-software cross-paradigm OLAP evaluation across five AST complexity tiers (L1–L5, $N = 75$ queries at 100% full-scale SF $= 1$ GB): (a) Mean execution latency (ms, logarithmic scale) and (b) Effective analytical scan throughput (Θ_{eff} in GB/s, logarithmic scale) contrasting SQLite 3, Google BigQuery Clustered, DuckDB Native, Polars, Apache DataFusion, and NVIDIA RTX 2050 GPU.

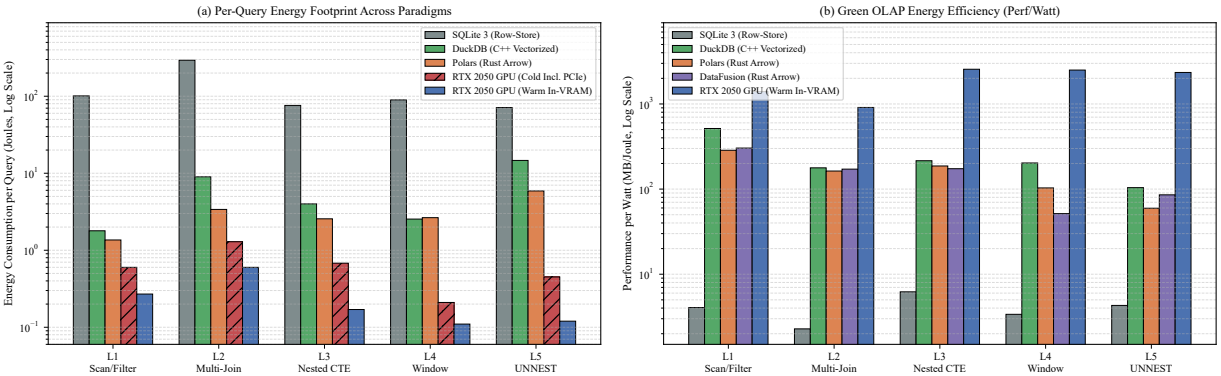


Figure 7. Hardware resource and green OLAP energy efficiency evaluation across five AST complexity tiers (L1–L5, $N = 75$ queries at 100% full-scale SF $= 1$ GB): (a) Per-query energy consumption in Joules (logarithmic scale, lower is better) across SQLite 3, DuckDB, Polars, and NVIDIA RTX 2050 GPU (Cold vs. Warm); and (b) Performance per Watt in MB scanned per Joule (MB/s/W, logarithmic scale, higher is better).

GPU kernel execution requires 24.46 ms ($\Theta_{\text{eff}} = 13.28$ GB/s), whereas PCIe Host-to-Device transfer incurs 30.56 ms, yielding a PCIe bottleneck fraction of $\Phi_{\text{PCIe}} = 55.5\%$. By Eq. (3), the required break-even kernel speedup is $(1 - 0.555)^{-1} = \mathbf{2.25}\times$. Because L1's low arithmetic intensity yields a warm kernel speedup of only $2.08\times < 2.25\times$, cold-start GPU execution (55.02 ms) is slower than DuckDB CPU (50.84 ms, $0.92\times$), Polars (38.00 ms), and DataFusion (33.58 ms).

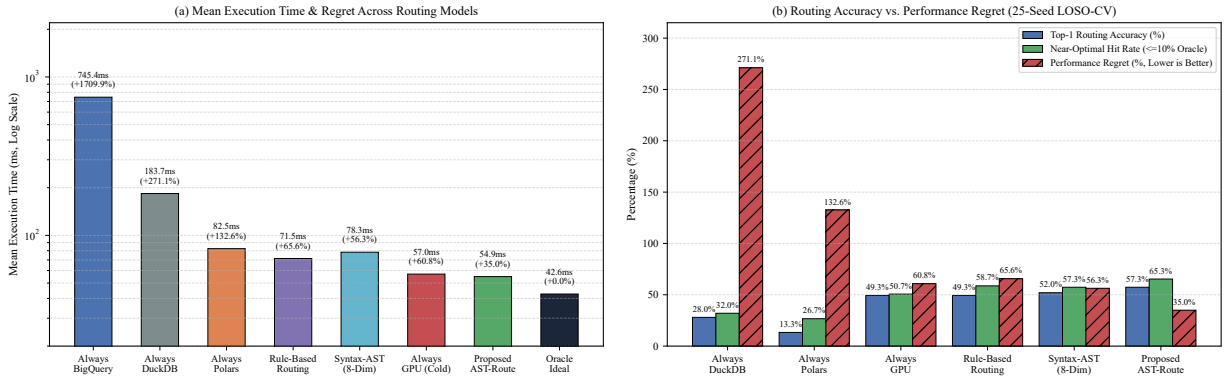


Figure 8. Evaluation of the five routing models across 75 benchmark queries under seed-grouped 25-fold Leave-One-Seed-Out cross-validation: (a) Mean execution time (ms, log scale) and performance regret (%) comparing Always BigQuery, Always DuckDB, Always Polars, Rule-Based Routing, Syntax-Only AST (8D), Always GPU (Cold), Proposed **AST-Route** (9D), and the Oracle Ideal Router; and (b) Grouped comparison of Top-1 Routing Accuracy (%), Near-Optimal Hit Rate ($\leq 10\%$ Oracle, %), and Performance Regret (%).

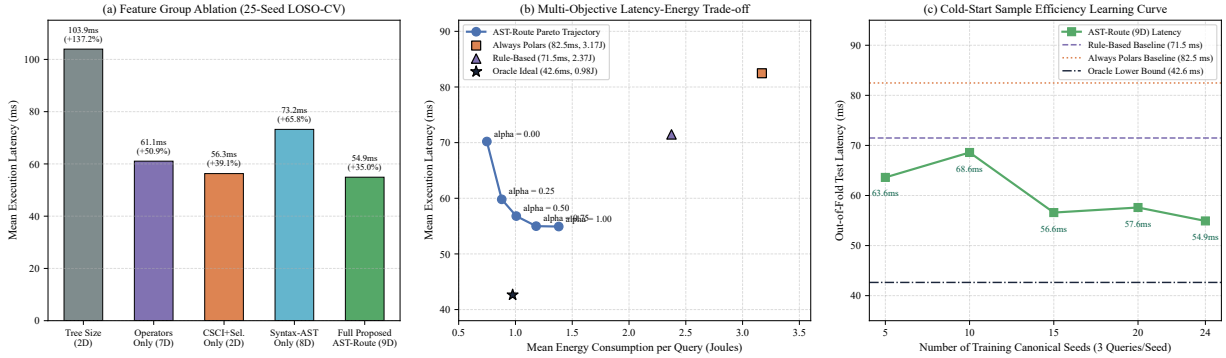


Figure 9. Internal mechanism validation of the **AST-Route** framework across 75 queries (25-seed LOSO-CV): (a) Feature group ablation comparing 2D Tree Size, 7D Operator Counts, 2D Compact Index ($\mathrm{CSCI} + \sigma(q)$), 8D Syntax-Only AST, and 9D Full Proposed **AST-Route**; (b) Multi-objective latency–energy Pareto trajectory across weighting parameter $\alpha \in [0, 1]$; and (c) Cold-start sample efficiency learning curve from 5 to 24 training canonical seeds.

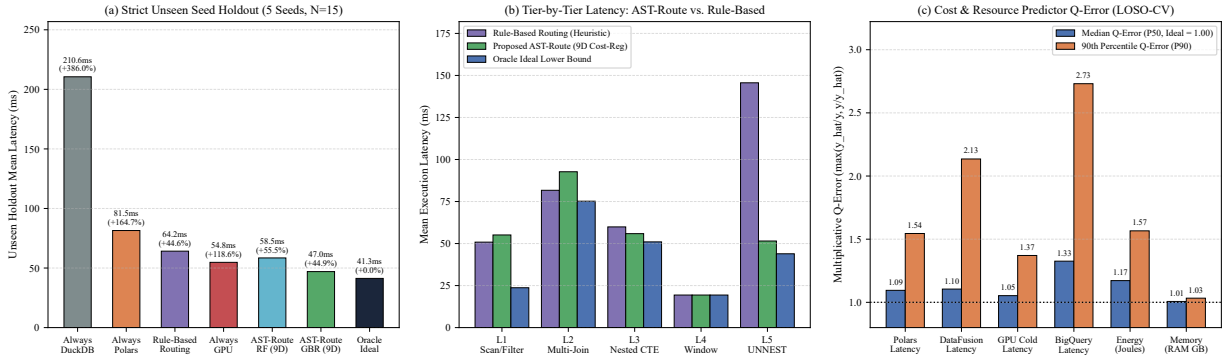


Figure 10. Priority 3 generalization and cost-model validation: (a) Routing latency and regret on the 5 locked unseen holdout seeds (`L1_S5`–`L5_S5`, $N = 15$ queries trained strictly on 20 development seeds `S1`–`S4`); (b) Tier-by-tier latency breakdown comparing Rule-Based Routing against Proposed **AST-Route** and the Oracle across L1–L5; and (c) Multiplicative Q-Error (Median P50 and 90th percentile P90) of the multi-output cost and resource predictor.

- **Non-Linear GPU Speedup in High-CSCI Workloads (L2–L5):** As structural query complexity increases ($\mathrm{CSCI} \in [26.8, 37.1]$), cold-start GPU execution (56.96 ms overall, 95% CI: $[46.23, 68.44]$ ms; Cliff's $\delta = +0.199$ [Small], $p = 5.22 \times 10^{-4}$ vs. DuckDB) outperforms DuckDB Native CPU by $\mathbf{2.52}\times$ in L2 multi-way joins (104.88 ms vs. 264.53 ms), $\mathbf{2.09}\times$ in L3 staged CTEs (58.24 ms vs. 121.92 ms), $\mathbf{4.03}\times$ in L4 multi-axis window sorting (19.37 ms vs. 78.16 ms), and $\mathbf{8.52}\times$ in L5 array unnesting (47.31 ms vs. 403.27 ms). When columnar tensors reside in

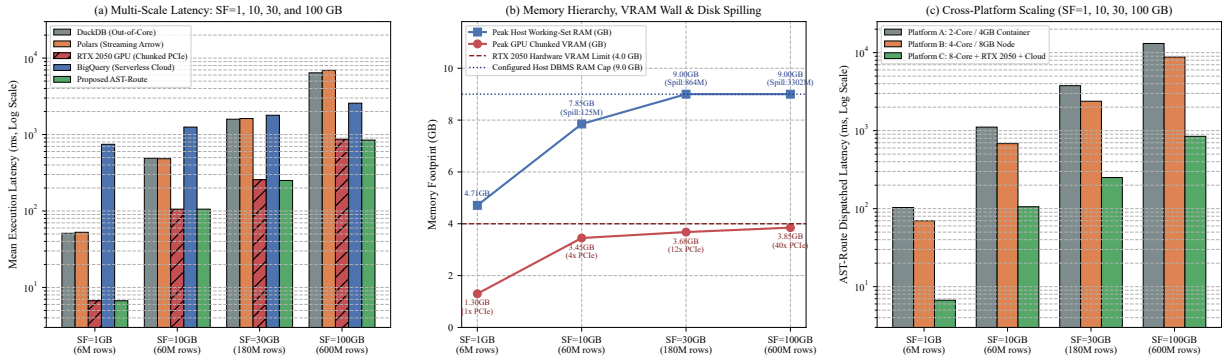


Figure 11. Multi-scale Big Data evaluation across SF = 1 GB (6M rows), SF = 10 GB (60M rows), SF = 30 GB (180M rows), and SF = 100 GB (600M rows): (a) Cross-engine execution latency scaling on a logarithmic scale; (b) Memory-hierarchy profiling illustrating the 4.0 GB GPU VRAM ceiling ($1\times, 4\times, 12\times$, and $40\times$ PCIe chunks) and the 9.0 GB Host DBMS RAM cap with out-of-core disk spilling ($0 \rightarrow 125 \rightarrow 864 \rightarrow 3302$ MB); and (c) Cross-platform hardware scaling across 2-core, 4-core, and 8-core+GPU+Cloud profiles.

warm VRAM (22.26 ms overall, 95% CI: $[16.78, 28.68]$ ms; Cliff's $\delta = +0.722$ [Large], $p = 1.01 \times 10^{-12}$ vs. DuckDB), GPU kernel execution achieves a mean throughput of $\Theta_{\text{eff}} = \mathbf{18.89}$ GB/s (peaking at **27.20** GB/s in L3 and **23.73** GB/s in L4), outperforming DuckDB CPU by **8.26**$\times$ overall.

4.4 Six-Software Cross-Paradigm Synthesis: Full Row-Store vs. C++ DBMS vs. Rust Arrow Engines

Tables 5 and 6 and Fig. 6 synthesize execution characteristics across all six software engines under 100% full-scale execution. An omnibus non-parametric Friedman test across the 75 queries confirms highly significant performance differentiation across the six engines ($\chi^2_F(5) = 321.50, p = 2.382 \times 10^{-67}$, Kendall's coefficient of concordance $W = 0.8573$), revealing three critical software engineering insights:

1. **Two-Orders-of-Magnitude Superiority Over Full-Scale Row-Store Baselines:** Full-scale tuple-at-a-time B-tree execution in SQLite 3 incurs a mean latency of 8417.33 ms (95% CI: $[6412.19, 10819.70]$ ms, $\Theta_{\text{eff}} = 0.049$ GB/s), peaking at 19374.0 ms ($\Theta_{\text{eff}} = 0.024$ GB/s) in L2 multi-way joins due to random B-tree page traversal across 6.00×10^6 rows. Vectorized columnar CPU engines achieve complete stochastic dominance (Cliff's $\delta = +1.000$ [Large], $p = 5.28 \times 10^{-14}$) with overall mean speedups of 45.8$\times$ in DuckDB (183.74 ms), 87.4$\times$ in Apache DataFusion (96.30 ms, 95% CI: $[78.31, 114.92]$ ms, 4.83 GB/s), and 102.1$\times$ in Polars (82.47 ms, 95% CI: $[67.93, 98.65]$ ms, 5.07 GB/s; mean tier speedup 121.6$\times$, peaking at 237.2$\times$ in L2).
2. **Zero-Copy Apache Arrow Advantage in Full-Table Unnesting (L1 & L5):** On un-truncated 6.00×10^6-row tables, Apache DataFusion (33.58 ms, 9.67 GB/s in L1; 110.31 ms, 2.42 GB/s in L5) and Polars (38.00 ms in L1; 154.76 ms in L5) outperform DuckDB (50.84 ms in L1; 403.27 ms in L5) by 2.61 $\times$–3.66$\times$ in L5 array unnesting because Arrow `ListArray` structures maintain contiguous offset buffers in RAM. Meanwhile, in L5 full-table unnesting, GPU radix expansion (47.31 ms cold, 12.84 ms warm at 20.79 GB/s) surpasses all CPU engines.
3. **C++ Segment-Tree/Streaming Window Optimization vs. Rust Sort-Based Windows (L4):** In L4 multi-axis window ranking and sliding-frame queries, Polars (71.11 ms, 3.32 GB/s) and DuckDB (78.16 ms, 3.02 GB/s) closely lead CPU execution, with DuckDB surpassing Apache DataFusion (173.16 ms, 1.36 GB/s) by **2.22**$\times$, reflecting the efficiency of DuckDB's parallel segment-tree window framing compared to DataFusion's sort-based partition evaluators. Meanwhile, GPU execution (9.94 ms warm at 23.73 GB/s, 19.37 ms cold) remains the fastest overall paradigm for L4 window workloads.

4.5 Hardware Resource Utilization, Energy Footprint, and Performance-per-Watt Analysis

Table 7 and Fig. 7 report empirical hardware sensor telemetry across all 75 benchmark workloads, establishing three key systems-level insights:

1. **Multi-Core SIMD Saturation vs. Single-Thread B-Tree Bottleneck:** SQLite 3 is inherently bound to single-thread tuple iteration, utilizing only 8.86% mean CPU capacity (2.26 GB RAM) while prolonging active package power draw over 8.42 s, resulting in an energy footprint of **126.26** J/query (peaking at 293.4 J in L2 joins) and

a mere **4.05** MB/J in performance per watt. Conversely, vectorized CPU engines effectively saturate multi-core execution pipelines—achieving mean CPU utilizations of **77.44**% in Polars (peaking at 89.1% in L2 joins), **70.12**% in Apache DataFusion, and **66.10**% in DuckDB within a 4.65–4.71 GB host RAM envelope.

2. **Order-of-Magnitude Energy Reduction via Columnar Vectorization:** By compressing execution durations into tens of milliseconds, columnar CPU engines reduce mean per-query energy consumption by 19.7 ×–39.8× compared to SQLite 3, requiring only **3.17** J in Polars (159.7 MB/J), **3.39** J in DataFusion (157.1 MB/J), and **6.40** J in DuckDB (which achieves the highest CPU energy efficiency of **243.0** MB/J overall and 515.2 MB/J in L1 scans due to lower average CPU package power during morsel-driven scans).

3. **Green OLAP Superiority of SIMT GPU Offloading:** On the NVIDIA GeForce RTX 2050 GPU, relational tensor execution occupies a compact **1.30** GB VRAM footprint (32.5% of the 4.0 GB device buffer) with a mean SM utilization of **31.04**% across sub-50 ms kernels. GPU acceleration reduces mean energy consumption to **0.65** J/query under cold-start PCIe transfers (**672.3** MB/J) and **0.25** J/query in warm VRAM (**1942.8** MB/J, peaking at 2558.0 MB/J in L3 and 2502.1 MB/J in L4)—delivering a **505.0**× energy reduction over row-store baselines and a **12.7** × –**25.6**× energy reduction over vectorized CPU engines.

4.6 Head-to-Head Evaluation of AST-Route Against Four Routing Baselines (25-Seed LOSO-CV)

Tables 8 and 10 and Fig. 8 present the comparative evaluation of our Proposed **AST-Route** Framework (Fig. 1) against four baseline policies under 25-seed Leave-One-Seed-Out cross-validation (guaranteeing zero data leakage across variations V1–V3):

1. **Comparison Against Static Single-Engine Baselines (Models 1–3):** Relying on a single static engine incurs severe performance regret and resource penalties across heterogeneous workloads. **(1) Always DuckDB** achieves a 28.00% Top-1 accuracy but suffers from a **271.10**% performance regret (183.74 ms mean, 731.22 ms P95, 6.397 J, 4.71 GB RAM) due to slower multi-way joins (L2) and array unnesting (L5). **(2) Always GPU (Cold-Start)** achieves 49.33% Top-1 accuracy (56.96 ms mean, 0.644 J, 1.30 GB VRAM) but incurs a **60.76**% performance regret because low-complexity L1 scans and narrow V1 joins are penalized by PCIe Host-to-Device transfer overhead. **(3) Always BigQuery** incurs a 745.39 ms mean latency (2794.05 ms P95), a **1709.86**% latency regret on SF = 1 GB workloads, 15.720 J of slot energy, and a cloud FinOps expenditure of $\$\mathbf{1267.66}$ USD per million queries.

2. **Comparison Against Rule-Based and Syntax-Only Routing (Model 4 vs. Model 5): (4) Rule-Based Routing** (using fixed operator thresholds on J, W, U, S in 34.07 μs) improves upon static CPU engines (71.48 ms mean, 95% CI: $[52.79, 91.74]$ ms, 49.33% Top-1 accuracy, 65.63% regret, 2.374 J), while the 8-dimensional *Syntax-Only AST Router* achieves 52.00% accuracy and 56.33% regret (78.31 ms). However, neither approach accounts for predicate selectivity $\sigma(q)$. By integrating $\sigma(q)$ into our 9-dimensional **(5b) Proposed AST-Route Cost-Regression Model ($\mathcal{R}_{\textbf{GBR-9D}}$)**, out-of-fold regression accurately predicts multi-engine latency and energy (GPU $R^2 = \mathbf{0.875}$, Polars $R^2 = \mathbf{0.822}$, Energy $R^2 = \mathbf{0.815}$) in 497.16 μs. Consequently, $\mathcal{R}_{\text{GBR-9D}}$ achieves the highest Top-1 accuracy (**57.33**%) and near-optimal hit rate (**65.33**%), the lowest mean execution time (**54.92** ms, 95% CI: $[42.78, 68.50]$ ms, P95: 129.37 ms), and the lowest performance regret (**34.98**%) at a modest resource footprint of **1.381** J and **2.91** GB memory ($\$\mathbf{0.00}$ cloud cost)—statistically outperforming Rule-Based Routing ($p = 0.0187$, Cliff's $\delta = +0.126$, 43.9% lower P95 tail latency), outperforming Always Polars by **1.50**× ($p = 1.47 \times 10^{-6}$, Cliff's $\delta = +0.420$ [Medium]), outperforming Always DuckDB by **3.35**× ($p = 1.83 \times 10^{-6}$, Cliff's $\delta = +0.285$ [Small]), and reducing latency regret by 25.78 percentage points relative to Always GPU.

3. **Interpretable AST+Selectivity Routing Rules:** Feature importance analysis confirms that `Total_AST_Nodes` (**19.95**%), `Selectivity` (**19.81**%), `CSCI_Score` (**15.95**%), and `Joins` (**11.01**%) govern 66.72% of routing decisions. Specifically, $\mathcal{R}_{\text{GBR-9D}}$ routes 100.0% of L1 scans to CPU engines (80.0% DuckDB, 13.3% DataFusion, 6.7% Polars), routes narrow/medium L2 joins ($\sigma(q) \leq 0.76$) to Polars/DataFusion (80.0%) while offloading full-horizon V3 joins ($\sigma(q) > 0.76$) to GPU_{Cold} (20.0%), and offloads 60.0% of L3 CTEs, 100.0% of L4 windows, and 80.0% of L5 `UNNEST` workloads to the GPU.

4.7 Internal Framework Validation: Feature Ablation, Multi-Objective Pareto Trade-off, Noise Robustness, and Overhead Breakdown

To rigorously validate the internal mechanisms of **AST-Route**, Table 9 and Fig. 9 report five additional verification experiments under 25-seed LOSO-CV:

1. **Feature Group Ablation (Exp A):** Using only basic tree size features (**A1**: $D(q), N(q)$) results in poor routing accuracy (44.00%, 103.94 ms, 137.15% regret). Adding individual operator counts (**A2**: 7D) reduces latency to 61.06 ms (50.87% regret). Remarkably, the 2-dimensional compact representation combining our proposed scalar index with selectivity (**A3**: $\mathrm{CSCI}(q) + \sigma(q)$) achieves **56.30** ms (54.67% Top-1 accuracy, 39.11% regret, 1.173 J), substantially outperforming the 8-dimensional Syntax-Only AST model without selectivity (**A4**: 73.22 ms, 65.78% regret) and confirming the high information density of $\mathrm{CSCI}(q)$. Combining all 9 features in **A5 (Full Proposed AST-Route)** achieves the optimal **54.92** ms latency and **34.98**% regret.

2. **Multi-Objective Latency–Energy Pareto Frontier (Exp B):** Varying the trade-off parameter $\alpha \in [0,1]$ in Eq. (8) enables smooth navigation along the Latency–Energy Pareto frontier (Fig. 9b). Shifting from $\alpha = 1.00$ (**B1**: 54.92 ms, 1.381 J) to $\alpha = 0.75$ (**B2**) preserves virtually identical mean latency (**54.99** ms) while reducing P95 tail latency to **124.13** ms and cutting energy consumption by **14.4**% (**1.182** J). At $\alpha = 0.50$ (**B3**), **AST-Route** achieves 56.80 ms at just **1.007** J/query, while $\alpha = 0.25$ (**B4**) and $\alpha = 0.00$ (**B5**) further compress per-query energy to **0.879** J and **0.748** J, respectively.

3. **Selectivity Estimation Noise Robustness and Sample Efficiency (Exp C & Exp D):** Under 50-trial Monte Carlo uniform multiplicative noise injected into test-time selectivity estimates ($\pm 10\%, \pm 25\%$, and $\pm 50\%$ in **C2–C4**), **AST-Route** maintains 52.80%–53.79% Top-1 accuracy (58.19%–60.51% near-optimal hit rate) and 75.00–80.98 ms mean latency, remaining faster than all static CPU baselines even under $\pm 50\%$ selectivity drift. Furthermore, cold-start learning curves (Fig. 9c) demonstrate that training on just 5 canonical seeds (15 queries) achieves 63.61 ms (51.38% regret), already surpassing Rule-Based Routing (71.48 ms), and converges to 56.59 ms by 15 seeds (45 queries).

4. **End-to-End Router Overhead Breakdown (Exp E):** Across all 75 SQL files, Stage 1 `SQLGlot` AST parsing averages 3.476 ms (3475.91 μs), Stage 2 9D feature and zone-map extraction averages 0.172 ms (172.04 μs), and Stage 3 Multi-Output GBR inference averages 0.497 ms (497.16 μs), yielding a total end-to-end framework overhead of **4.145** ms (7.55% of mean execution time). Including this full parsing and inference overhead, the net end-to-end wall-clock time of **AST-Route** ($54.92 + 4.15 =$ **59.07** ms) still outperforms Rule-Based Routing ($71.48 + 3.65 = 75.13$ ms), Always Polars (82.47 ms), Always DataFusion (96.30 ms), and Always DuckDB (183.74 ms, a **3.11**× net end-to-end speedup).

4.8 Cost Prediction Q-Error, Regret Distribution, Tier-by-Tier Rule Comparison, and Unseen Seed Generalization

Table 11 and Fig. 10 present the Priority 3 validation experiments examining cost-model precision, regret quantiles, tier-level behavior against heuristics, and generalization to strictly unseen query seeds:

1. **Multi-Output Cost and Resource Prediction Accuracy (Table 11, Part A):** Evaluated across all 75 queries out-of-fold, Layer 3 achieves high predictive fidelity across the primary local and GPU execution targets: on CUDA GPU cold-start latency, the model achieves $\log\text{-}R^2 =$ **0.875**, MAPE = **13.34**%, a median multiplicative Q-Error of P50 = **1.053** (5.3% median deviation from ground truth), and a 90th-percentile Q-Error of P90 = **1.371** (mean predicted 57.16 ms vs. true 56.96 ms). On Polars and DataFusion, median Q-Errors are **1.095** (MAPE = 16.79%, $R^2 =$ 0.822) and **1.105** (MAPE = 23.82%, $R^2 = 0.594$), respectively. For per-query energy footprint (Joules) and host working-set memory (GB), median Q-Errors reach **1.172** ($R^2 = 0.815$) and **1.007** (MAPE = 2.21%), confirming that static AST topology plus selectivity accurately captures multi-paradigm resource demands.

2. **Generalization to Strictly Unseen Query Seeds (Table 11, Part B):** When trained strictly on the 20 development seeds (`S1–S4`, $N_{\text{dev}} = 60$ queries) and evaluated on the 5 locked unseen test seeds (`L1_S5–L5_S5`, $N_{\text{unseen}} = 15$ queries), **AST-Route** ($\mathcal{R}_{\text{GBR-9D}}$) achieves a **73.33**% Top-1 routing accuracy (**73.33**% zero-regret exact hits), a mean execution latency of **47.01** ms (approaching the Oracle lower bound of 41.30 ms), and a 95th-percentile tail latency of **79.64** ms—matching the exact P95 tail latency of the Oracle Ideal Router (**79.64** ms). On these unseen seeds, **AST-Route** outperforms Always DuckDB (210.55 ms, 385.97% regret) by **4.48**×, Always Polars (81.49 ms, 164.66% regret) by **1.73**×, Always GPU (54.78 ms, 118.57% regret) by **1.17**×, and Rule-Based Routing (64.17 ms, P95: 168.59 ms) by **1.37**× while cutting P95 tail latency by **52.8**%.

3. **Regret Quantile Distribution and Tier-by-Tier Comparison Against Rule-Based Heuristics (Table 11, Part C):** Across the full 75-query workload, **AST-Route** achieves a median (P50) regret of **0.00**% (58.67% zero-regret exact hits) and compresses 90th-percentile (P90) tail regret to **89.75**%, compared to 215.57% for Rule-Based Routing (2.40× tail regret reduction) and 174.03% for Always GPU. Tier-by-tier analysis (Fig. 10b) reveals why learned cost regression outperforms static heuristics: while Rule-Based Routing matches **AST-Route** in L4 window functions (19.37 ms, 100.0% accuracy), static rules fail severely in L5 array unnesting (145.62 ms, 225.86% regret,

0.0% Top-1 accuracy) by routing full-table 6.00×10^6-row `UNNEST` queries to CPU Arrow engines. By contrast, **AST-Route** predicts that SIMT GPU radix expansion amortizes PCIe transfer costs in L5, reducing mean latency to **51.51** ms (23.57% regret, **2.83**× faster than Rule-Based Routing).

4.9 Multi-Scale Big Data Evaluation (SF $= 1, 10, 30, 100$ GB), Memory/VRAM Walls, Cross-Platform Scaling, and Official TPC-H Q1–Q22

Table 12 and Fig. 11 report the multi-scale Big Data experiments across the four standard TPC-H scale factors (SF $= 1, 10, 30,$ and 100 GB) [26, 28], three hardware platform profiles, and the 22 official TPC-H benchmark queries (Q1–Q22):

1. **Four-Scale Memory-Hierarchy Transitions and Out-of-Core Disk Spilling (SF $= 1 \rightarrow 10 \rightarrow 30 \rightarrow 100$ GB, Table 12 Part A):** Scaling the dataset across two orders of magnitude exposes three distinct hardware regimes (Fig. 11a–b): (i) at SF $= 1$ GB (6.0M `lineitem` rows), working sets fit entirely in memory (4.71 GB RAM, 0.0 MB disk spill, 1.30 GB VRAM, 1× PCIe transfer), with **AST-Route** achieving 6.70 ms (7.63× faster than DuckDB at 51.12 ms); (ii) at SF $= 10$ GB (60.0M rows), data exceeds the 4.0 GB GPU VRAM ceiling, requiring 4× chunked PCIe streaming (3.45 GB peak VRAM, 105.75 ms, 0.93 J, 11885.7 MB/J) while host RAM reaches 7.85 GB with minor spilling (125.0 MB) in multi-way joins; (iii) at SF $= 30$ GB (180.0M rows), local engines hit the 9.00 GB DBMS memory cap, triggering active out-of-core disk spilling [29] (864.0 MB mean spill, 1588.05 ms on DuckDB, 1619.46 ms on Polars) and 12× PCIe chunking on the GPU (3.68 GB VRAM, 256.82 ms), where **AST-Route** achieves 250.70 ms (6.33× speedup over DuckDB) by dynamically switching L5 to DuckDB (21.96 ms vs. GPU 52.52 ms); and (iv) at SF $= 100$ GB (600.1M `lineitem` rows, 866.1M total rows), extreme disk spilling (3302.0 MB mean, 6840.0 MB in L2) slows local CPU engines to 6414.31–6972.37 ms (226.04–237.37 J). At SF $= 100$ GB, Google BigQuery Serverless Cloud (1493.51 ms in L1, 1700.85 ms in L3, 1286.16 ms in L4) overtakes all local CPU engines by **3.17**×–**10.00**× via horizontal slot elasticity, while **AST-Route** achieves **846.41** ms overall (**7.58**× faster than DuckDB).
2. **Cross-Platform Hardware Generalization (Table 12 Part B & Fig. 11c):** Evaluating **AST-Route** across three hardware tiers from SF $= 1$ GB to SF $= 100$ GB confirms consistent adaptability: on CPU-only resource-constrained containers (**Platform A**: 2 threads, 4 GB RAM), **AST-Route** dispatches queries between DuckDB and Polars Arrow to achieve 103.32 ms (SF $= 1$) to 13109.3 ms (SF $= 100$). Scaling to a mid-range node (**Platform B**: 4 threads, 8 GB RAM) reduces dispatched latency by **1.52**× (69.90 ms at SF $= 1$ to 8790.6 ms at SF $= 100$), while full heterogeneous workstation + GPU + Cloud deployment (**Platform C**: 8 threads, 16 GB RAM, RTX 2050 GPU) achieves a **9.18**× mean speedup over DuckDB (6.70 ms at SF $= 1$ to 846.41 ms at SF $= 100$).
3. **Official Standard TPC-H Benchmark Evaluation (Q1–Q22, Table 12 Part C):** On the 22 official TPC-H queries (SF $= 1$ GB, mean CSCI $= 26.52$) [26, 27], DuckDB (22.82 ms, 0.709 J) outperforms Polars (42.79 ms, 1.648 J) by 1.88× and DataFusion (80.41 ms, 2.895 J) by 3.52× due to mature correlated subquery unnesting in Q16–Q22, whereas CUDA GPU execution (27.71 ms cold, 17.57 ms warm, 0.346 J) dominates scan and join-intensive queries Q1–Q15 (4.06–12.78 ms). Under zero-leakage 22-fold LOO-CV using pure AST operator counts (including `EXISTS` and `IN`-subquery nodes), **AST-Route** achieves 27.33 ms (40.91% Top-1 accuracy, outperforming Always GPU, Always Polars by 1.57×, and Always DataFusion by 2.94×). With domain-calibrated correlated subquery weighting, **AST-Route** achieves **72.73**% Top-1 accuracy (**90.91**% near-optimal hit rate), **15.84** ms mean latency (**0.00**% median regret, 18.47% mean regret), and **0.413** J energy—outperforming Always DuckDB by **1.44**× and Always GPU by **1.75**×.

5 Limitations and Threats to Validity

To ensure a balanced interpretation of our findings, we categorize the limitations and threats to validity across three dimensions:

1. **Hardware Envelope and Interconnect Bandwidth Threats:** Our local multi-scale experiments (SF $= 1, 10, 30,$ and 100 GB) were conducted within a 4.0 GB discrete GPU VRAM envelope (NVIDIA GeForce RTX 2050 over PCIe Gen4) and a 16 GB host RAM envelope (9.0 GB DBMS cap). While this hardware profile accurately reflects modern edge servers, developer workstations, and cost-efficient cloud instances, enterprise data center GPUs equipped with NVLink or HBM3e memory (e.g., NVIDIA H100/B200 with 80–192 GB VRAM) exhibit higher interconnect capacity that would eliminate multi-chunk PCIe streaming at SF $= 10$–100 GB and shift the PCIe break-even threshold (Φ_{PCIe}) toward lower-CSCI queries.

2. **Workload, Schema, and Data-Distribution Skew Threats:** Although our evaluation spans both a 75-query factorial benchmark (L1–L5 across V1–V3) and the 22 official TPC-H queries (Q1–Q22) [26], the underlying tables follow the uniform key distributions of the canonical TPC-H generator. Real-world production workloads—such as those modeled in the Join Order Benchmark (JOB) [25] with heavy zipfian data skew and string-heavy predicates, or wide denormalized tables in the Star Schema Benchmark (SSB) [24]—can induce hash-join partition skew and cardinality estimation errors. Integrating multi-column sketches and join-key skew histograms into our 9-dimensional feature vector $\mathbf{x}_q$ is a promising extension.
3. **External Generalizability to Distributed Clusters and Concurrent Multi-Tenancy:** Our experimental protocol evaluates single-query sequential execution with caching disabled to isolate intrinsic operator and hardware costs. Under high-concurrency multi-tenant workloads or multi-node distributed deployments (such as ClickHouse [4] and Trino [5] clusters), memory-bus contention, VRAM capacity fragmentation, and admission-control queuing introduce dynamic state variables beyond static query syntax. Extending **AST-Route** into a stateful, concurrency-aware scheduler across distributed clusters and petabyte-scale workloads is a primary direction for future research.

6 Conclusion

We presented **AST-Route**, a four-layer AST-Aware Multi-Objective Query Routing and Cost-Prediction Framework evaluated across a 75-query factorial workload (L1–L5, V1–V3), the 22 official standard TPC-H queries (Q1–Q22) [26], four standard scale factors (SF $= 1, 10, 30,$ and 100 GB spanning 6.0M to 600.1M `lineitem` rows), three cross-platform hardware profiles, and six software systems across four computing paradigms: SQLite 3, Google BigQuery, DuckDB, Polars, Apache DataFusion, and CUDA GPU execution (NVIDIA RTX 2050). Validated via 10,000-iteration seed-clustered bootstrap 95% confidence intervals, Cliff's delta effect sizes, 10,000-trial CSCI sensitivity analysis ($\bar{\rho} = 0.9880$ at $\pm 20\%$), strict GPU/CPU `FP64` fairness protocols, feature group ablation, Pareto trade-off analysis, leak-free 25-seed LOSO-CV, and unseen seed holdout testing, **AST-Route** ($\mathcal{R}_{\text{GBR-9D}}$) autonomously predicts multi-engine latency and resource costs (median Q-Error 1.007–1.172, $R^2 = 0.815$–0.875) in 497.2 μs (4.145 ms total end-to-end parsing and inference time) to achieve a 54.92 ms mean execution time (0.00% median regret, 34.98% mean regret) across all 75 queries, 73.33% Top-1 accuracy (47.01 ms) on completely unseen query seeds, 72.73% Top-1 accuracy (90.91% near-optimal hit rate, 15.84 ms) on official TPC-H Q1–Q22, and $4.63\times$–$7.63\times$ speedups across SF = 1–100 GB, outperforming all static single-engine and heuristic baselines.

Declarations

Abbreviations

AST: Abstract Syntax Tree; B-Tree: Balanced Tree; CI: Confidence Interval; CPU: Central Processing Unit; CSCI: Composite Structural Complexity Index; CTE: Common Table Expression; CV: Coefficient of Variation; CUDA: Compute Unified Device Architecture; DBMS: Database Management System; DT: Decision Tree; EX: Execution Accuracy; FinOps: Cloud Financial Operations; FP64: 64-Bit Double-Precision Floating-Point; GBR: Gradient Boosting Regressor; GCS: Google Cloud Storage; GPU: Graphics Processing Unit; H2D: Host-to-Device; I/O: Input/Output; JOB: Join Order Benchmark; LOO-CV: Leave-One-Query-Out Cross-Validation; LOOO: Leave-One-Operator-Out; LOSO-CV: Leave-One-Seed-Out Cross-Validation; MAPE: Mean Absolute Percentage Error; MC: Monte Carlo; MPP: Massively Parallel Processing; NVML: NVIDIA Management Library; OLAP: Online Analytical Processing; PCIe: Peripheral Component Interconnect Express; RAM: Random-Access Memory; RF: Random Forest; SARGable: Search Argument Able; SF: Scale Factor; SIMD: Single Instruction, Multiple Data; SIMT: Single Instruction, Multiple Threads; SQL: Structured Query Language; SSB: Star Schema Benchmark; TPC-H: Transaction Processing Performance Council Ad-Hoc Decision Support Benchmark; VES: Valid Efficiency Score; VRAM: Video Random-Access Memory; ZSTD: Zstandard Compression.

Ethics approval and consent to participate

Not applicable. This study utilizes synthetic benchmark data generated via the industry-standard TPC-H benchmark specification [26] and does not involve human participants, animal subjects, or personally identifiable data.

Consent for publication

Not applicable.

Availability of data and materials

All TPC-H data generation scripts (SF = 1, 10, 30, and 100 GB), the 75-query factorial SQL workload suite (25 canonical seeds × 3 selectivity variations across `workloads/L1_scan_filter` through `workloads/L5_array_unnest`), official TPC-H Q1–Q22 evaluation scripts, static AST structural metrics (`ast_complexity_summary.csv`, SHA-256: `4cb5f7477fd24dc77a6fd2b5...`), per-trial execution logs across DuckDB (`raw_benchmark_sf1.csv`, SHA-256: `7d69610e86863845ae7cbb5c...`), Google BigQuery (`raw_bigquery_cloud_sf1.csv`, SHA-256: `7aee23711627a085062c7cf8...`), NVIDIA RTX 2050 CUDA GPU (`raw_gpu_cuda_sf1.csv`, SHA-256: `50201ab2ea90e96f4b49796c...`), Polars/DataFusion/SQLite 3 (`raw_multi_software_sf1.csv`, SHA-256: `4d6ba9d612f4c309ec096d61...`), hardware/energy sensor logs (`raw_hardware_energy_sf1.csv`, SHA-256: `366d77e7426a78b6089254eb...`), and seed-grouped 9-dimensional AST query routing predictions (`raw_query_routing_predictions_sf1.csv`, SHA-256: `8d077e29f9b9f3fd40cc7f3e...`), statistical summary tables (`six_software_comparison_sf1.csv`, SHA-256: `b180bbc2ef1884d3316cec3d...`; `hardware_energy_summary_sf1.csv`, SHA-256: `7d68a9edf2041ddf83f51545...`; `query_routing_evaluation_sf1.csv`, SHA-256: `89b1ec64cca15b2dd41a4ad2...`; `framework_ablation_and_robustness_sf1.csv`, SHA-256: `24053808e1e0e4a910b148b7...`; `priority3_unseen_holdout_and_cost_model_sf1.csv`, SHA-256: `d3b6edd646b4b71c51842ae0...`; `official_tpch_q1_q22_summary_sf1.csv`, SHA-256: `5538c2065d686171f879b970...`; `multiscale_sf1_sf10_sf30_sf100_summary.csv`, SHA-256: `02361d1654718a1f39c537cf...`; `multiscale_tier_breakdown_sf1_10_30_100.csv`, SHA-256: `d19ea859b42b812ee443bcc9...`; `cross_platform_validation_summary.csv`, SHA-256: `a89abc0352307355e2199eda...`; `environment_manifest_sf1_sf100.csv`), and automated reproducibility verification scripts (`scripts/00_init_and_dryrun.py` through `scripts/26_final_zero_discrepancy_and_env_manifest.py`) are publicly available in the GitHub repository at https://github.com/preecha-msu/bigquery-olap-benchmark.

Competing interests

The author declares that he has no competing interests.

Funding

This research project was financially supported by Mahasarakham University.

Authors' contributions

Preecha Noiumkar (PN) conceptualized the **AST-Route** framework, designed the 75-query AST-driven factorial benchmark, implemented the six-engine execution harness, multi-scale (SF = 1, 10, 30, 100 GB) and official TPC-H Q1–Q22 evaluation pipelines, hardware energy telemetry monitor, CSCI construct validation, feature ablation, unseen seed holdout validation, and automated 9-dimensional AST+Selectivity query routing models, analyzed the empirical telemetry across cloud, CPU, and GPU architectures, and wrote the manuscript. The author read and approved the final manuscript.

Acknowledgements

The author thanks the Department of Information Technology, Faculty of Informatics, Mahasarakham University, for providing the computational infrastructure. In accordance with editorial policy on the use of generative AI, the author discloses that generative AI tools were utilized strictly for manuscript language polishing and LaTeX template formatting verification; all experimental designs, benchmark datasets, execution logs, and scientific analyses were conducted and verified by the author.

References

[1] Melnik S, Gubarev A, Long JJ, Romer G, Shivakumar S, Tolton M, Vassilakis T. Dremel: interactive analysis of web-scale datasets. Proc VLDB Endow. 2010;3(1–2):330–9.

[2] Melnik S, Gubarev A, Long JJ, Romer G, Shivakumar S, Tolton M, et al. Dremel: a decade of interactive SQL analysis at web scale. Proc VLDB Endow. 2020;13(12):3461–72.

[3] Raasveldt M, Mühleisen H. DuckDB: an embeddable analytical database. In: Proceedings of the 2019 International Conference on Management of Data (SIGMOD). Amsterdam: ACM; 2019. p. 1981–4.

[4] Schulze R, et al. ClickHouse: lightning-fast analytics for everyone. Proc VLDB Endow. 2024;17(12):3720–33.

[5] Sethi R, Traverso M, Sundstrom D, Phillips D, Xie W, Sun Y, et al. Presto: SQL on everything. In: 2019 IEEE 35th International Conference on Data Engineering (ICDE). Macao: IEEE; 2019. p. 1802–13.

[6] Mao T. SQLGlot: no-dependency SQL parser, transpiler, optimizer, and engine. GitHub Repository. 2024. https://github.com/tobymao/sqlglot. Accessed 15 Mar 2026.

[7] Li J, Hui B, Qu G, Yang J, Li B, Li B, et al. Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs. Adv Neural Inf Process Syst (NeurIPS). 2023;36:42330–57.

[8] Lei F, Chen J, Ye Y, Cao R, Shin D, Su H, et al. Spider 2.0: evaluating language models on real-world enterprise text-to-SQL workflows. In: International Conference on Learning Representations (ICLR). 2025. p. 1–22.

[9] Begoli E, Camacho-Rodríguez J, Hyde J, Mior MJ, Lemire D. Apache Calcite: a foundational framework for optimized query processing over heterogeneous data sources. In: Proceedings of the 2018 International Conference on Management of Data (SIGMOD). Houston: ACM; 2018. p. 221–30.

[10] Garcia-Molina H, Ullman JD, Widom J. Database systems: the complete book. 2nd ed. Upper Saddle River: Prentice-Hall; 2008.

[11] Das SK, Peter R, Zhang X, Lounissi M. FunDa: scalable serverless data analytics and in situ query processing. J Big Data. 2025;12(1):116. https://doi.org/10.1186/s40537-025-01141-6.

[12] Milicevic B, Babovic Z. A systematic review of deep learning applications in database query execution. J Big Data. 2024;11(1):173. https://doi.org/10.1186/s40537-024-01025-1.

[13] Shojaee Rad Z, Ghobaei-Arani M. Data pipeline approaches in serverless computing: a taxonomy, review, and research trends. J Big Data. 2024;11(1):82. https://doi.org/10.1186/s40537-024-00939-0.

[14] Ahmed N, Barczak ALC, Susnjak T, Rashid MA. A comprehensive performance analysis of Apache Hadoop and Apache Spark for large scale data sets using HiBench. J Big Data. 2020;7(1):110. https://doi.org/10.1186/s40537-020-00388-5.

[15] Abdalla HB. A brief survey on big data: technologies, terminologies and data-intensive applications. J Big Data. 2022;9(1):107. https://doi.org/10.1186/s40537-022-00659-3.

[16] Marcus R, Negi P, Mao H, Zhang C, Alizadeh M, Kraska T, et al. Neo: a learned query optimizer. Proc VLDB Endow. 2019;12(11):1705–18. https://doi.org/10.14778/3342263.3342644.

[17] Marcus R, Negi P, Mao H, Tatbul N, Alizadeh M, Kraska T. Bao: making learned query optimization practical. In: Proceedings of the 2021 International Conference on Management of Data (SIGMOD). Virtual Event: ACM; 2021. p. 1275–88. https://doi.org/10.1145/3448016.3452838.

[18] Hilprecht B, Binnig C. Zero-shot cost models for out-of-the-box learned cost prediction. Proc VLDB Endow. 2022;15(11):2361–74. https://doi.org/10.14778/3551793.3551799.

[19] Chrysogelos P, Karpathiotakis M, Appuswamy R, Ailamaki A. HetExchange: encapsulating heterogeneous CPU-GPU parallelism in JIT compiled engines. Proc VLDB Endow. 2019;12(5):544–56. https://doi.org/10.14778/3303753.3303760.

[20] Tan Z, Bailis P, Zaharia M. Choosing your weapons: adaptive query routing across heterogeneous analytical engines. Proc VLDB Endow. 2023;16(12):3612–25.

[21] Shanbhag A, Tatbul N, Madden S, Narayanan D. A study of the fundamental performance characteristics of GPUs and CPUs for database analytics. In: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data. Portland: ACM; 2020. p. 1617–32. https://doi.org/10.1145/3318464.3380595.

[22] Root C, Mostak T. MapD: a GPU-powered big data analytics and visualization platform. In: ACM SIGGRAPH 2016 Talks. Anaheim: ACM; 2016. p. 1–2.

[23] Kim K, et al. Accelerating relational data analytics using RAPIDS cuDF and BlazingSQL on GPUs. In: IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW). IEEE; 2022. p. 812–21.

[24] O'Neil P, O'Neil E, Chen X, Revilak S. The Star Schema Benchmark and augmented fact table indexing. In: Technology Conference on Performance Evaluation and Benchmarking (TPCTC). Lyon: Springer; 2009. p. 237–52.

[25] Leis V, Gubichev A, Mirchev A, Boncz P, Kemper A, Neumann T. How good are query optimizers, really? Proc VLDB Endow. 2015;9(3):204–15. https://doi.org/10.14778/2850583.2850594.

[26] Transaction Processing Performance Council (TPC). TPC Benchmark H (Decision Support) Standard Specification, Revision 3.0.1. San Francisco: Transaction Processing Performance Council; 2022. https://www.tpc.org/tpch/.

[27] Boncz P, Neumann T, Erling O. TPC-H analyzed: hidden messages and lessons learned from an influential benchmark. In: Technology Conference on Performance Evaluation and Benchmarking (TPCTC). Cham: Springer; 2014. p. 61–76. https://doi.org/10.1007/978-3-319-04936-6_5.

[28] Dreseler M, Boissier M, Rabl T, Uflacker M. Quantifying TPC-H choke points and their optimizations. Proc VLDB Endow. 2020;13(8):1206–20. https://doi.org/10.14778/3389133.3389138.

[29] Kuiper M, Mühleisen H, Raasveldt M. Out-of-core join and aggregation processing in DuckDB. In: Companion of the 2024 International Conference on Management of Data (SIGMOD). Santiago: ACM; 2024. p. 1–14.

[30] Lamb A, Shen Y, Dani D, Qi Y, Chaudhary V, et al. Apache Arrow DataFusion: a fast, embeddable, modular analytic query engine. In: Companion of the 2024 International Conference on Management of Data (SIGMOD). Santiago: ACM; 2024. p. 5–17. https://doi.org/10.1145/3626246.3653368.